Google Summer of Code

BLT NEXT

Md Kaif Ansari

March 2026

Table of Contents

1. Introduction
    - Personal Details
    - About Me
    - Why BLT?
2. Community Engagement & Contributions
3. Background of the Project
    - Problem Statement
    - What Exists Today
    - Why MCP?
4. Project Overview
    - What Will Be Built
    - What Will Not Be Built
5. Technical Design
    - Architecture Overview
    - MCP Server Layers (Resources, Tools, Prompts)
    - Transport Strategy (stdio vs HTTP/SSE)
    - Authentication Design
    - Integration with BLT Backend
6. Implementation Plan
    - Week-by-Week Schedule (12 Weeks)
7. Testing Strategy
8. Documentation Plan
9. Why Me?
10. Other Commitments

1. Introduction

Personal Details

Name: Md Kaif Ansari

GitHub: mdkaifansari04

Slack Handle: [redacted]

Email: [redacted]

Phone: [redacted]

Location: India (IST, UTC +5:30)

Degree: B.Tech in Computer Science

University: Aliah University

Year: 3rd Year, Batch 2024–2027

About Me

I'm Md Kaif Ansari, a full-stack developer mainly working with Python, Django, TypeScript, and Node.js. I enjoy working on systems that sit close to infrastructure APIs, protocol layers, AI integrations, and developer tooling.

I started contributing to BLT in late 2025 and since then I've been involved across multiple parts of the ecosystem: the Django core backend, the Cloudflare Worker-based API layer, browser extension, and supporting repositories. So far I've contributed 55+ merged PRs across more than 13+ BLT repositories.

Because of this, I'm approaching BLT-Next from theory. I understand how BLT-core works, and we are transforming every core module into independent submodules, over which I've a good experience of the schema and overall integration, I also have worked on BLT-API which clears up my understanding of Cloudflare workers and D1 database, where the constraints are (D1, Workers, migrations, schema ownership), and how contributors interact with the system.

Why BLT-Next?

BLT is not just a bug logging platform. It combines security research, gamification, community collaboration, and now, a full architectural reset. That combination makes it structurally interesting from a systems perspective.

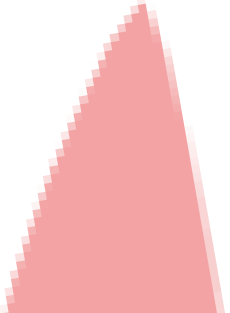
For GSoC 2026, the direction toward a static, edge-first, contributor-friendly ecosystem is the logical next step. Django served its purpose. But when the monolith becomes the bottleneck, for contributors, for performance, for maintainability the right move is a clean break, not a patch.

Right now, BLT's architecture is heavy.

With BLT-Next, BLT becomes lightweight by default.

That shift matters. It turns BLT from:

> *"a platform that requires a full Django environment to contribute to"* into *"a platform where anyone can open an HTML file, make a change, and ship it in five minutes"*



2. Community Engagement & Contributions

I have been actively contributing to the OWASP BLT ecosystem since November 2025. Below is a categorized summary of my first 39 merged pull requests across the BLT organization.

NOTE: The below section has been developed with the help of AI (Claude). [Proof of contribution]

Contributions by Category

#	Category	Repository	PR	Description
1	Bug Fix	BLT-Action	#74	Fix: Prevent Auto-Unassignment When PR is Created Within 24 Hours
2	Bug Fix	BLT	#4683	Fix: Filter out users with empty usernames in leaderboard view
3	Refactor	BLT	#4586	Refactor: Organize the docs module
4	Bug Fix	BLT-Action	#75	Fix: Search qualifiers correction
5	Testing	BLT	#4752	Fix: Correct test file placement for Django test discovery
6	Bug Fix	BLT	#4750	Fix: Pre-commit failure due to missing height and width attributes

7	Maintenance	BLT	#4756	Fix: Remove deprecated DEFAULT_FILE_STORAGE setting (Django upgrade)
8	UI	BLT	#4916	Fix: Leaderboard Sidebar Dark Mode Background
9	DevOps	BLT	#4767	Fix: Pin Postgres version to 17.6 (avoid breaking changes)
10	Refactor	BLT	#4587	Refactor: Organize scripts into dedicated scripts/ directory
11	Bug Fix	BLT	#4919	Fix: Organization Create Failing Due to CharField Limit
12	Feature	MY-GSOC-TOOL	#3	Feat: Add Runtime Editing Mode for GSoC Dashboard
13	Feature	BLT-Sammich	#47	Feat: Add Sammich Landing Page
14	Refactor	BLT-Lettuce	#106	Refactor: Update Complete Branding of the Homepage
15	Bug Fix	BLT	#5365	Fix: Filter bot username from global leaderboard
16	Maintenance	BLT	#5398	Chore: Add myself under mentee

17	UI	BLT	#5002	Refactor: Fix Dark Mode UI Issues on Organization Dashboard
18	Refactor	BLT-GSoC-Ideas	#19	Refactor: Updated Project F
19	Maintenance	BLT-GSoC-Ideas	#9	Chore: Add project-f
20	Refactor	BLT-Leaf	#23	Fix: Standardize database name to pr_tracker across config files
21	UI	BLT-Leaf	#30	UI: Add Dark Theme
22	Feature	BLT-Hackathon	#8	Feat: Add Review Leaderboard, Fix Incorrect Leaderboard Data
23	Bug Fix	BLT-API	#4	Fix: Relative import error, add lock to gitignore
24	UI	BLT-Leaf	#28	UI: Update UI to Align with BLT-Core Branding
25	Bug Fix	BLT	#5530	Fix: Improve notification text visibility in dark mode
26	Feature	BLT-API	#7	UI: Update Landing Page
27	Feature	BLT-Design	#3	Feat: Add BLT Design System

28	Feature	BLT-Extension	#45	Feat: Update and Improve BLT Extension
29	Feature	BLT-API	#10	Feat: Add D1 Support for Domain Route
30	Feature	BLT-API	#13	D1: Add Bugs API Schema Migration and Local Seed Updates
31	Feature	BLT-API	#15	Feat: Add Remote DB Setup Script
32	Refactor	BLT-API	#18	Refactor: Update Issue APIs to Bugs
33	Feature	BLT-NetGuardian	#8	UI: Enhance UI and Functionality for BLT-NetGuardian
34	Feature	BLT-Rewards	#22	Refactor: Migrate to Cloudflare Worker and Updated UI
35	Feature	BLT-API	#20	Refactor: Migrate User Schema to D1 and Updated Migration Scripts
36	Refactor	BLT-Leaf	#183	Refactor: Update Schema SQL into Migration SQL Files

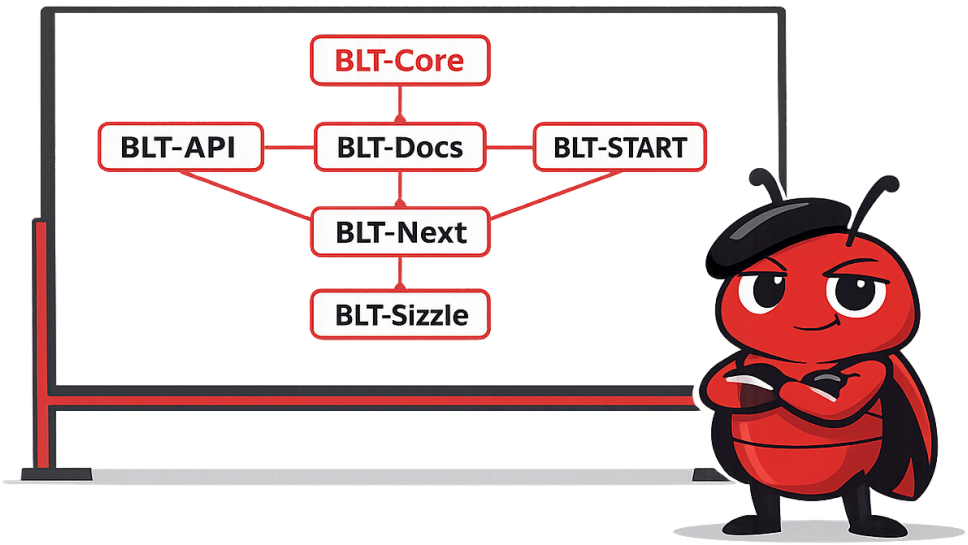
37	Feature	BLT-API	#25	Feat: Extract Email Templates into Separate HTML Files
38	Feature	BLT-API	#21	Feat: Implement Auth with 2FA Verification + Mailgun Email
39	UI	BLT-Next	#9	UI: Design Update and Add Logo

Contribution Highlights

BLT Core (Django) — Fixed real production bugs including leaderboard data integrity, organization creation failures, dark mode regressions, and deprecated Django settings. Demonstrates deep familiarity with the BLT Django codebase.

BLT-API (Cloudflare Workers + D1) — One of the most active contributors to this repo. Implemented the full D1 database migration for the bugs schema, added domain routing, remote DB setup scripts, user schema migration, 2FA email auth, and Cloudflare Worker deployment. [proof of work]

Multi-Repo Breadth — Contributed across 13+ BLT repositories (BLT, BLT-API, BLT-Leaf, BLT-Lettuce, BLT-Sammich, BLT-Extension, BLT-Design, BLT-Hackathon, BLT-NetGuardian, BLT-Rewards, BLT-Next, BLT-Action, MY-GSOC-TOOL), showing system-level understanding of the full BLT ecosystem exactly what BLT-Next needs to expose all of it through one unified interface.



3. Background of the Project

What Exists Today

OWASP BLT started as a Django monolith; one big repo handling everything: bug reporting, leaderboard, user auth, company dashboards, and BLT rewards. For a long time that worked. But as BLT grew into something bigger than just bug logger it now has hackathons, jobs, newsletters, developer tools, community dashboards, the monolith became the bottleneck.

The BLT Core repo is now archived. That's not a failure, that's a decision. The team recognized that maintaining a Django app with this many responsibilities wasn't the right path forward. Contributor onboarding was painful; you needed a full Python environment, database setup, Django migrations, the works, just to fix a UI bug. Deployment was slow. Global response times were whatever Heroku or the VPS gave you on that day.

The ecosystem had already started fragmenting naturally; BLT-Extension, BLT-Leaf, BLT-Action, BLT-Rewards all lived as separate repos because they had to. The monolith couldn't absorb them cleanly.

What BLT-Next Is

BLT-Next is the architectural reset. The idea is simple: every piece of BLT that can be static, should be static. GitHub Pages serves the frontend; plain HTML, Tailwind via CDN, HTMX or JS for dynamic interactions. No build step, no bundler, no framework overhead. When something genuinely needs server-side logic auth, database writes, external API calls, that's a Cloudflare Python Worker. Each submodule is its own GitHub Pages site with its own worker if needed. One ecosystem, modular pieces.

The v0 landing page is already live. Dark mode works. The design foundation is there. But here's the thing Donnie made clear when I asked about scope directly on Slack.BLT-Next isn't just "implement all the submodules." It's specifically for things that BLT-Pages and BLT-API cannot handle. The clearest example is anonymous bug reporting. BLT-Pages is static, it can't handle identity-free submissions with tracking tokens. BLT-API is authenticated, it assumes a logged-in user. BLT-Next is the layer that sits between the static frontend, but with a Worker that handles the cases neither of the other two can. That's the real scope of this project. Not a submodule factory, but a thoughtful implementation of features that genuinely have nowhere else to live in the current BLT architecture.

Why This Architecture Makes Sense

A few things make this approach genuinely better for BLT specifically, not just "modern is good":

Contributor friction drops to near zero. Want to fix something in BLT-Jobs? Open the HTML file, make your change, push. No virtualenv, no migrations, no `pip install -r requirements.txt`. The barrier to a first contribution goes from an hour to five minutes.

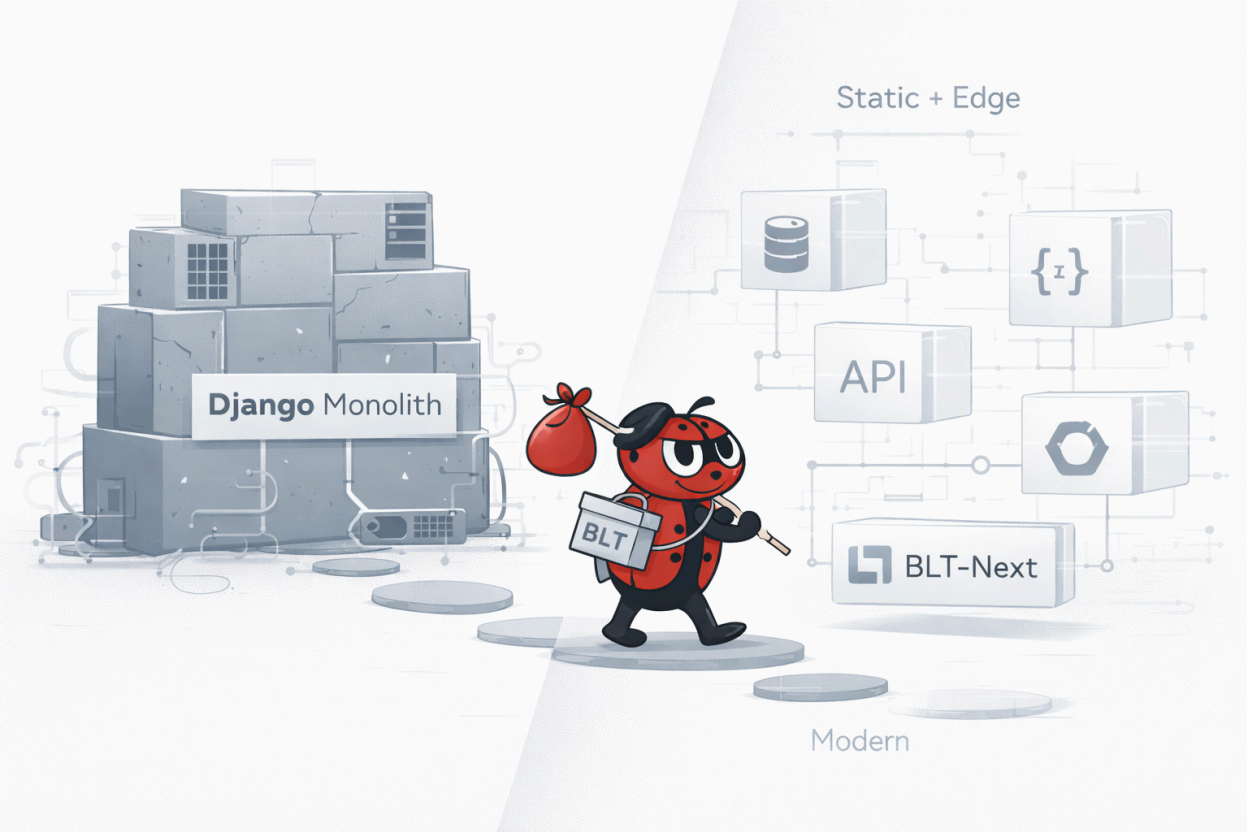
Global performance by default. Static assets on Cloudflare's CDN + Python Workers at the edge means sub-200ms responses without any optimization work(most importantly our cost is down to $0 after using all free services). The Django monolith couldn't touch that without significant infrastructure investment.

Each submodule is independently deployable. BLT-Hackathons breaking doesn't take down BLT-Jobs. A worker update to BLT-Newsletter doesn't require touching anything else. This is the kind of isolation the monolith never had.

The stack matches where open source contributors already are. HTML, vanilla JS, Python; these are the lowest-barrier technologies in the ecosystem. Cloudflare Workers have excellent free tier and docs. GitHub Pages is free and instant. The whole thing can run at scale for effectively zero infrastructure cost.

What GSoC Builds

The foundation exists. The vision is clear. What's left is the actual implementation; taking the ecosystem of submodules from concept to shipped. That means building out BLT-Monitor, BLT-Timer-Web, BLT-Sizzle, BLT-Byte, BLT-Zero, BLT-Panini, BLT-Vanini-Web, BLT-Hackathons, BLT-Jobs, BLT-Newsletter, BLT-Design, and BLT-Docs, each with working frontend, worker integrations where needed, proper testing, and published deployments. Not prototypes. Not v0 shells. Actual working tools that the BLT community can use.



4. Project Overview

What Will Be Built

The core question for BLT-Next scope, straight from a conversation with Donnie: what are the things BLT-Pages and BLT-API cannot handle? That's what gets built here.

BLT-Pages is static with no server-side logic, no dynamic state. BLT-API is authenticated; it assumes a logged-in user with a JWT. BLT-Next fills the gap: static frontend backed by Cloudflare Workers for features that need server-side logic but don't require authentication, or that need a fundamentally different UX than what BLT-API serves.

Flagship feature: Anonymous Bug Reporting

This is the clearest example of something BLT-Pages and BLT-API genuinely cannot do. No account, no login, no stored identity, just a bug report form, a Cloudflare Worker that assigns a random tracking token, and a D1 row. The reporter gets a token to check status. Reviewers see the report. Nobody sees who filed it. This is the core BLT-Next use case.

Beyond anonymous reporting, the submodules in scope are those that either need identity-free interactions or have a UX footprint too large for BLT-Pages to handle cleanly:

Submodule	Why It Can't Live in BLT-Pages or BLT-API	Worker Needed
Anonymous Bug Report	Needs server-side token generation, no auth	Yes

BLT-Monitor	Needs cron triggers, scheduled pings, uptime logs	Yes
BLT-Timer-Web	Needs per-user session persistence without full auth flow	Yes
BLT-Sizzle	Needs server-side aggregation and ranking logic	Yes
BLT-Zero	Needs submission handling and status tracking	Yes
BLT-Panini	Needs external API fetching and D1 caching layer	Yes
BLT-Vanish-Web	Needs request tracking without full user accounts	Yes
BLT-Hackathons	Too large for BLT-Pages, needs dynamic registration data	Yes
BLT-Jobs	Dynamic listings, submission form, filtering	Yes
BLT-Newsletter	Subscription handling, archive rendering	Yes
BLT-Byte	Mostly static, optional voting worker	Optional
BLT-Design	Pure static component library	No

BLT-Docs	Pure static documentation	No

tldr; *the filter for "does this belong in BLT-Next" is; can BLT-Pages serve it statically? Can BLT-API handle it with existing auth? If the answer to both is no, it lives here.*

Note: BLT Next doesn't have this feature in it, but it comes under the scope of BLT-Next proposal as all the modules are independent.

Each submodule follows the same pattern:

- Static frontend on GitHub Pages
- Cloudflare Python Worker if dynamic data is needed
- D1 database for persistence where required
- HTMX/JS for dynamic interactions, no framework overhead

On top of individual submodules, there's integration work the BLT-Next landing page needs to surface these submodules cohesively. Navigation, cross-submodule linking, consistent design language across all of them. That's not a small thing when you're talking about 12 separate deployments.

What Will NOT Be Built

Being honest about scope matters here. In 12 weeks, some things are explicitly out:

- **Django migration/compatibility layer** — Django is archived. There's no backward compatibility work. Clean break.
- **Mobile apps** — BLT-Leaf (mobile) has its own roadmap, not touching it here.
- **BLT-Extension rework** — browser extension has its own release cycle, out of scope.

- **Full admin dashboard** — basic content management through D1 is in scope, a full admin panel is not.
- **Real-time features** — WebSockets, live updates, anything requiring persistent connections. Cloudflare Workers are stateless by design, keeping it that way.
- **Remaining submodules not in the priority list** — BLT-CVE, BLT-Toasty, BLT-Sammich etc. are ecosystem pieces for future contributors.

Feature Parity Map

The Django monolith has features that need to land somewhere in BLT-Next. Here's how they map:

Django Feature	BLT-Next Equivalent	Lives In
Bug reporting form	Static form + Worker endpoint	BLT-Next core
User auth / sessions	JWT via Cloudflare Worker	BLT-Next core
Leaderboard	Static page + Worker API	BLT-Next core
Hackathon management	Full BLT-Hackathons submodule	BLT-Hackathons
Job listings	Full BLT-Jobs submodule	BLT-Jobs
Newsletter	Full BLT-Newsletter submodule	BLT-Newsletter
Site monitoring	Full BLT-Monitor submodule	BLT-Monitor

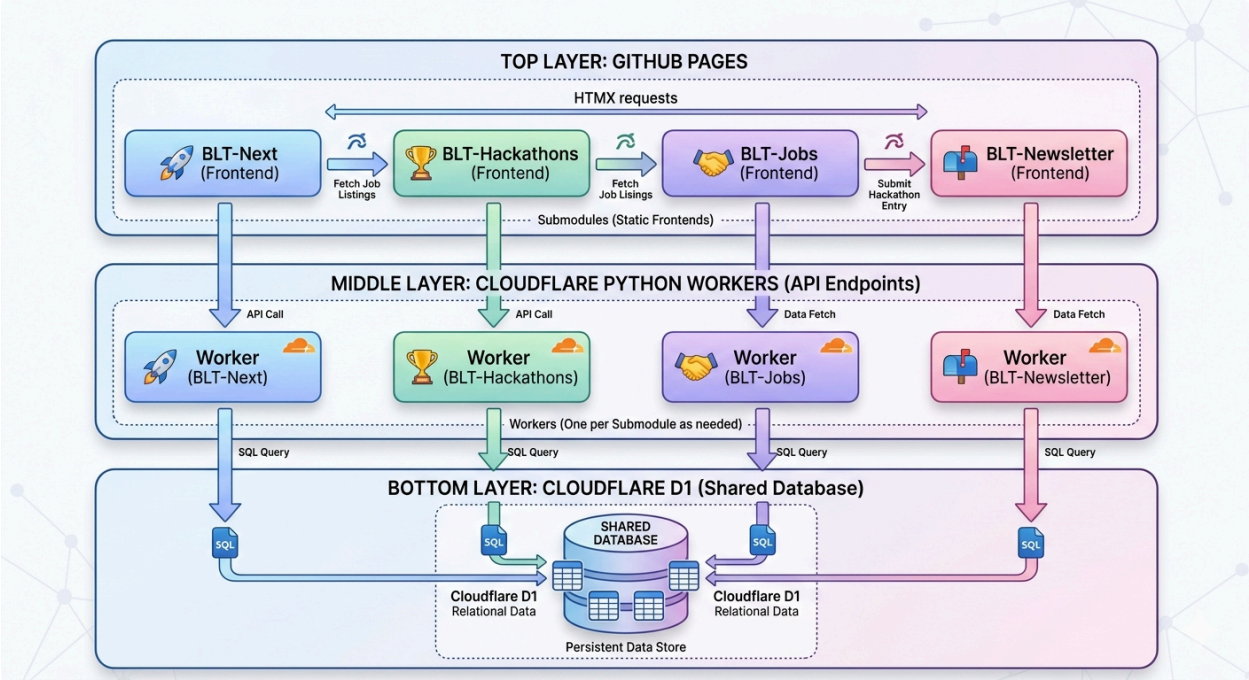
Design system	BLT-Design submodule	BLT-Design
Documentation	BLT-Docs submodule	BLT-Docs
Admin panel	D1 direct + minimal worker UI	BLT-Next core

***tldr**: everything Django was doing either maps cleanly to a submodule or gets handled by a Worker endpoint. Nothing gets dropped, it just lives in the right place now.*

5. Technical Design

Architecture Overview

The architecture has three layers. Static frontend on GitHub Pages, dynamic logic on Cloudflare Python Workers, persistence on D1. Every submodule follows this same pattern; the only variable is whether it needs a worker or not.



Frontend Layer

No build step. No bundler. No framework. The frontend is plain HTML with Tailwind via CDN and JS for dynamic interactions. The design system from BLT-Design gives every submodule a consistent visual language without copy-pasting CSS everywhere.

Every page follows progressive enhancement; core content loads and works without JavaScript. HTMX adds interactivity on top. Vanilla JS handles anything HTMX can't.

Here's what a typical submodule page looks like BLT-Jobs as the example:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>BLT Jobs — Security Job Board</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/htmx.org@1.9.10"></script>
  <link rel="stylesheet" href="/assets/css/blt-design.css" />
</head>
<body class="bg-gray-950 text-white">

  <!-- Jobs list — loaded dynamically via HTMX -->
  <div
    id="jobs-list"
    hx-get="https://blt-jobs.workers.dev/api/jobs"
    hx-trigger="load"
    hx-swap="innerHTML"
    class="max-w-4xl mx-auto px-4 py-8"
  >
    <p class="text-gray-400">Loading jobs...</p>
  </div>

  <!-- Submit a job posting -->
  <form
    hx-post="https://blt-jobs.workers.dev/api/jobs"
    hx-target="#jobs-list"
    hx-swap="afterbegin"
    class="max-w-4xl mx-auto px-4 py-4 space-y-4"
  >
    <input name="title" placeholder="Job Title"
      class="w-full bg-gray-800 rounded px-4 py-2" required />
    <input name="company" placeholder="Company"
      class="w-full bg-gray-800 rounded px-4 py-2" required />
    <input name="url" placeholder="Apply URL"
      class="w-full bg-gray-800 rounded px-4 py-2" required />
    <button type="submit"
      class="bg-orange-500 hover:bg-orange-600 px-6 py-2 rounded font-semibold">
      Post Job
    </button>
  </form>

</body>
</html>
```

No React. No Vue. No compilation. A contributor who knows basic HTML can read this, understand it, and change it in five minutes.

Backend Layer — Cloudflare Python Workers

Each submodule that needs dynamic logic gets its own Python Worker. The worker handles routing, validation, auth checks, and D1 queries. Here's the pattern using BLT-Jobs as the example:

```python
# workers/blt_jobs.py
from js import Response, Request
from workers import WorkerEntryPoint
import json

class Default(WorkerEntryPoint):
  async def on_fetch(request, env):
      url = request.url
      method = request.method

      # CORS headers for GitHub Pages origin
      headers = {
          "Content-Type": "application/json",
          "Access-Control-Allow-Origin": "https://owasp-blt.github.io",
          "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
          "Access-Control-Allow-Headers": "Content-Type, Authorization",
      }

      if method == "OPTIONS":
          return Response.new("", headers=headers, status=204)

      if "/api/jobs" in url:
          if method == "GET":
              return await get_jobs(env, headers)
          if method == "POST":
              return await create_job(request, env, headers)

      return Response.new(
          json.dumps({"error": "Not found"}),
          headers=headers,
          status=404
      )


async def get_jobs(env, headers):
    result = await env.DB.prepare(
        "SELECT * FROM jobs ORDER BY created_at DESC LIMIT 50"
    ).all()
    jobs = [dict(row) for row in result.results]
    return Response.new(json.dumps(jobs), headers=headers, status=200)


async def create_job(request, env, headers):
    data = await request.json()

    title = data.get("title", "").strip()
    company = data.get("company", "").strip()
    url = data.get("url", "").strip()

    if not title or not company or not url:
        return Response.new(
            json.dumps({"error": "Missing required fields"}),
            headers=headers,
            status=400
        )

    await env.DB.prepare(
        "INSERT INTO jobs (title, company, url, created_at) VALUES (?, ?, ?,
datetime('now'))"
    ).bind(title, company, url).run()

    return Response.new(
        json.dumps({"success": True}),
        headers=headers,
        status=201
    )
```

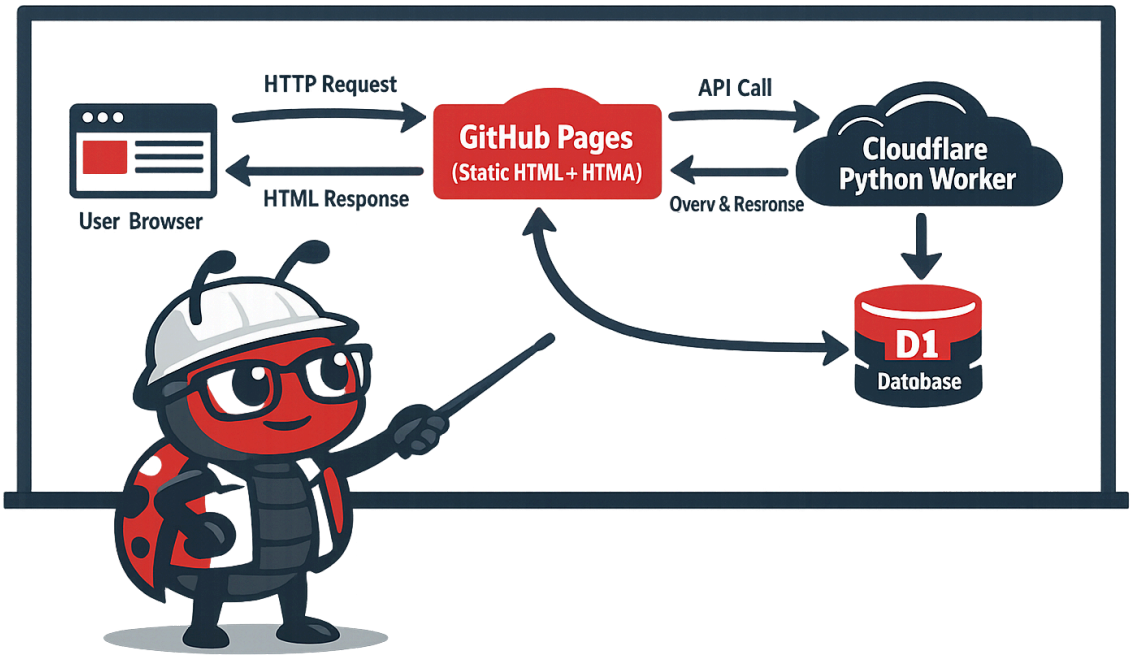
Same pattern across every worker — routing at the top, handlers below, CORS always set, D1 for persistence. Once you've read one worker you've read them all.

Database Layer — D1 Schema

Shared D1 instance, one schema file, all submodules use it. No separate databases per submodule would be a maintenance nightmare. Here's the schema for the priority submodules:

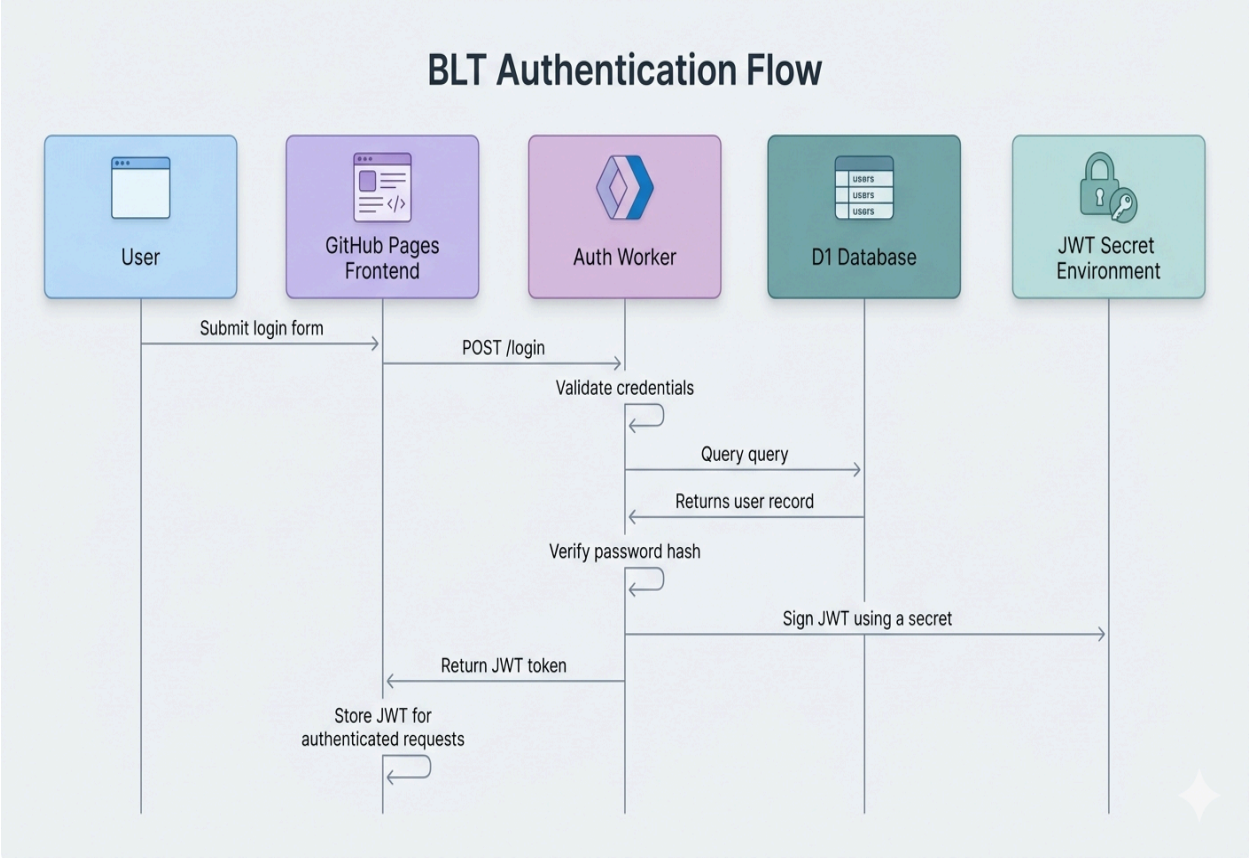
```sql
-- BLT-Jobs
CREATE TABLE IF NOT EXISTS jobs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    company TEXT NOT NULL,
    url TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now')),
    is_active INTEGER DEFAULT 1
);

-- BLT-Hackathons
CREATE TABLE IF NOT EXISTS hackathons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    start_date TEXT,
    end_date TEXT,
    registration_url TEXT,
    status TEXT DEFAULT 'upcoming',
    created_at TEXT DEFAULT (datetime('now'))
);

-- BLT-Newsletter
CREATE TABLE IF NOT EXISTS newsletter_subscribers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT UNIQUE NOT NULL,
    subscribed_at TEXT DEFAULT (datetime('now')),
    is_active INTEGER DEFAULT 1
);

CREATE TABLE IF NOT EXISTS newsletter_issues (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT,
    published_at TEXT,
    is_published INTEGER DEFAULT 0
);
```

```sql
-- BLT-Monitor
CREATE TABLE IF NOT EXISTS monitored_sites (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    url TEXT NOT NULL,
    last_checked TEXT,
    status TEXT DEFAULT 'unknown',
    uptime_percentage REAL DEFAULT 100.0
);

CREATE TABLE IF NOT EXISTS uptime_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    site_id INTEGER REFERENCES monitored_sites(id),
    checked_at TEXT DEFAULT (datetime('now')),
    status_code INTEGER,
    response_time_ms INTEGER,
    is_up INTEGER
);

-- BLT-Timer-Web
CREATE TABLE IF NOT EXISTS timer_sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT,
    label TEXT,
    started_at TEXT,
    ended_at TEXT,
    duration_seconds INTEGER
);
```



Auth Flow

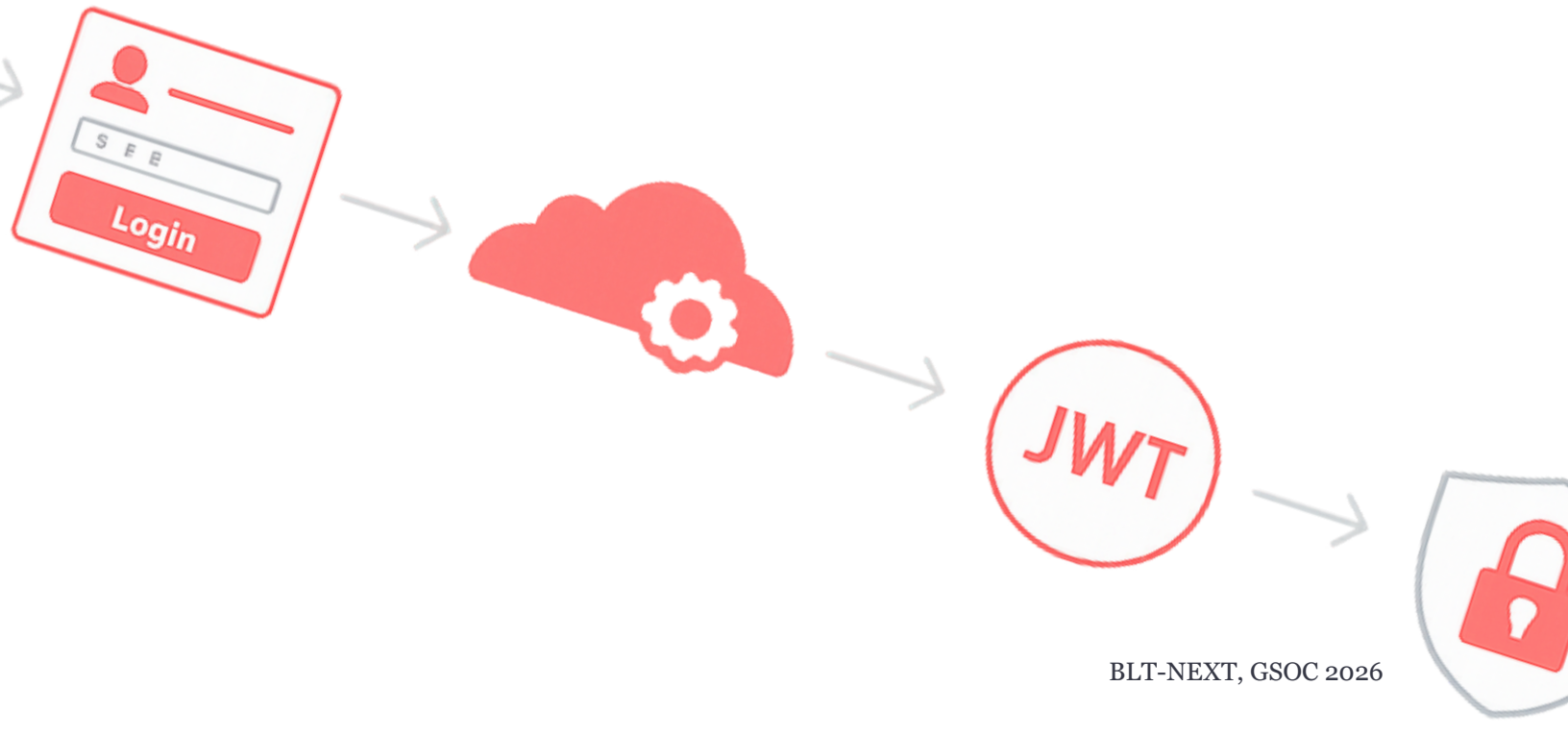
Django sessions are gone. Auth is JWT-based, handled entirely in the Worker layer. No cookies, no server-side sessions.



```python
# workers/auth.py
import json
import hmac
import hashlib
import base64
import time

def create_jwt(user_id: str, secret: str) -> str:
    header = (
        base64.urlsafe_b64encode(json.dumps({"alg": "HS256", "typ": "JWT"}).encode())
        .rstrip(b"=")
        .decode()
    )
    payload = (
        base64.urlsafe_b64encode(
            json.dumps(
                {
                    "sub": user_id,
                    "iat": int(time.time()),
                    "exp": int(time.time()) + 86400,  # 24 hours
                }
            ).encode()
        )
        .rstrip(b"=")
        .decode()
    )
    signature_input = f"{header}.{payload}".encode()
    signature = hmac.new(secret.encode(), signature_input, hashlib.sha256).digest()
    signature_b64 = base64.urlsafe_b64encode(signature).rstrip(b"=").decode()

    return f"{header}.{payload}.{signature_b64}"

def verify_jwt(token: str, secret: str) -> dict | None:
    try:
        header, payload, signature = token.split(".")
        expected_input = f"{header}.{payload}".encode()
        expected_sig = hmac.new(
            secret.encode(), expected_input, hashlib.sha256
        ).digest()
        expected_b64 = base64.urlsafe_b64encode(expected_sig).rstrip(b"=").decode()
        if not hmac.compare_digest(signature, expected_b64):
            return None
        decoded = json.loads(base64.urlsafe_b64decode(payload + "==").decode())

        if decoded.get("exp", 0) < int(time.time()):
            return None

        return decoded
    except Exception:
        return None
```

On the frontend, the JWT lives in a JS variable after login not localStorage, but sometimes cookie. It's passed in the Authorization header on every frontend request that needs auth:

```
// assets/js/auth.js
let currentToken = null;

async function login(email, password) {
    const res = await fetch('https://blt-auth.workers.dev/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
    });

    if (res.ok) {
        const { token } = await res.json();
        currentToken = token;
        // configure HTMX to send auth header on every request
        document.body.addEventListener('htmx:configRequest', (e) => {
            if (currentToken) {
                e.detail.headers['Authorization'] = `Bearer ${currentToken}`;
            }
        });
    }
}
```



CORS and Routing

Each worker has CORS locked to GitHub Pages origins. The `wrangler.toml` handles routing — each submodule worker gets its own route:

```
# wrangler.toml
name = "blt-next"
compatibility_date = "2024-01-01"

[[d1_databases]]
binding = "DB"
database_name = "blt-next-db"
database_id = "your-d1-id"

# Each submodule worker route
[[routes]]
pattern = "blt-jobs.workers.dev/*"
script = "workers/blt_jobs.py"

[[routes]]
pattern = "blt-hackathons.workers.dev/*"
script = "workers/blt_hackathons.py"

[[routes]]
pattern = "blt-monitor.workers.dev/*"
script = "workers/blt_monitor.py"

[[routes]]
pattern = "blt-newsletter.workers.dev/*"
script = "workers/blt_newsletter.py"
```

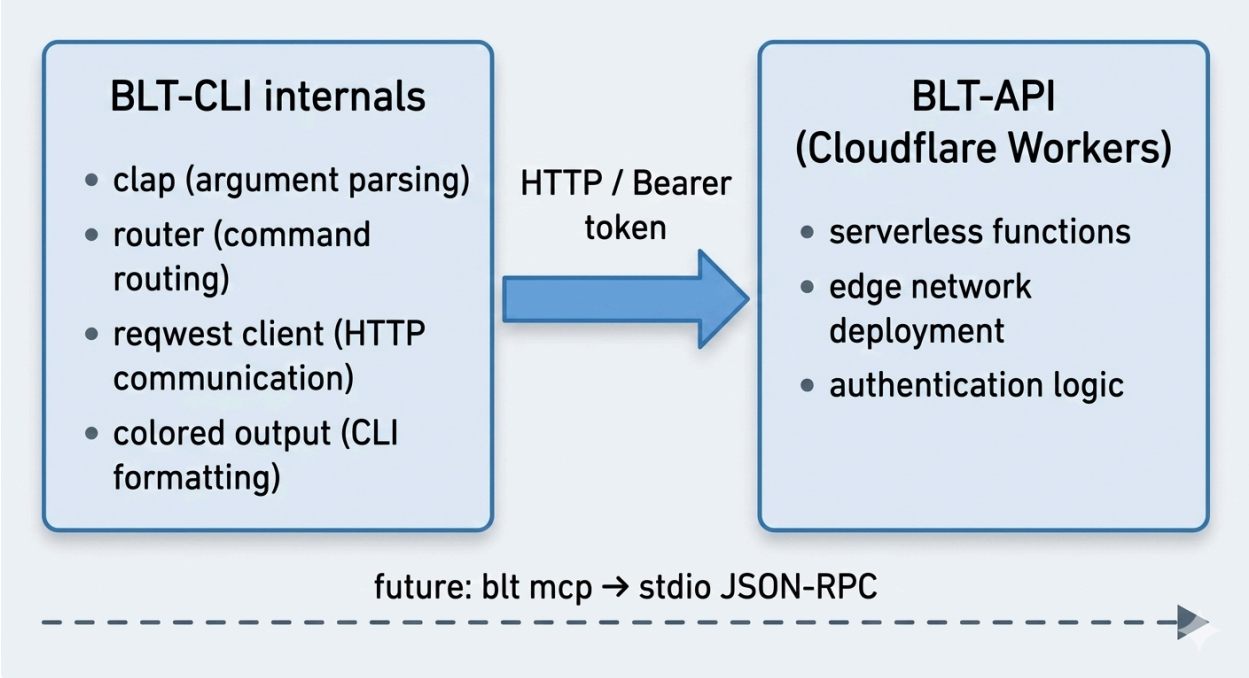
BLT-CLI (The detailed story)

A standalone Rust CLI that talks to BLT-API directly. Same backend as BLT-API, different interface this one's for humans in the terminal.

```
blt leaderboard          # top contributors, colored + ranked
blt issues               # recent bugs, filterable
blt issues --limit 25
blt whoami               # current user info
blt issue submit --title "XSS on login" --severity high
blt bacon award @kaif 50 "great find"
blt pr check 456         # BLT-Leaf readiness
blt scan trigger blt.owasp.org
blt config --api-key YOUR_KEY

```

Config lives at `~/.blt/config.toml` — API key + base URL, set once with `blt config`.

```
// startup banner — red to match the BLT mosquito logo
fn print_banner() {
    println!("{}", "
 ▄▄▄
```



NOTE: All the images except the screenshot and code block are generated by AI (Gemini nanobana and chatgpt) it may contain spelling mistakes please ignore. After multiple iterations, even the model is not good enough for minor spelling mistakes.

Already built a working Rust POC during the proposal phase, blt leaderboard, blt issues, blt whoami all hitting real BLT-API endpoints with colored ranked output and the BLT ASCII banner. Just for experimenting, and the below screenshot belongs to the POC..

BLT-CLI — Full Command Reference

```
AUTHENTICATION
  blt auth login              Authenticate with BLT API, store JWT locally
  blt auth logout             Clear stored credentials
  blt auth whoami             Show current authenticated user

BUGS
  blt bugs list               List recent bugs (paginated)
  blt bugs list --filter open List only open bugs
  blt bugs get <id>           Get full details of a specific bug
  blt bugs post "<title>"     Submit a new bug report
  blt bugs search "<query>"   Search bugs by keyword

LEADERBOARD
  blt leaderboard             Show current global leaderboard
  blt leaderboard --top 10    Show top N contributors
  blt leaderboard --monthly   Filter by current month

ORGANIZATIONS
  blt orgs list               List all organizations on BLT
  blt orgs get <slug>         Get details of a specific org
  blt orgs bugs <slug>        List bugs filed against an org

JOBS
  blt jobs list               List active security job postings
  blt jobs get <id>           Get full job details

PROFILE
  blt profile                 Show your BLT profile
  blt contributors list       List top contributors

TIMELOGS
  blt timelogs list           List your time tracking sessions
  blt timelogs start "<label>" Start a new session
  blt timelogs stop           Stop current session

SECURITY
  blt incidents list          List security incidents
  blt domain get <domain>     Get domain security info
  blt feed                    Show activity feed

REWARDS
  blt rewards                 Show your rewards and points
  blt rewards leaderboard     Show rewards leaderboard

CONFIGURATION
  blt config set-key <key>    Set your BLT API key
  blt config set-url <url>    Set custom API base URL
  blt health                  Check BLT API health status

GLOBAL FLAGS
  --json                      Output raw JSON (pipe-friendly)
  --no-color                  Disable colored output
  --verbose                   Show full HTTP request/response
  -h, --help                  Print help
  -V, --version               Print version
```

Working Screenshot of POC

```
 ) ~/code/open-source/owasp/blt-cli ) on  master ?5
blt bugs
```

```
BLT
OWASP BLT — Bug Logging Tool

ID    Domain     Status      Description
5     unknown    reviewing   Weak password reset mechanism
4     unknown    open        Missing rate limiting on API e
3     unknown    open        Unrestricted file upload allow
2     unknown    closed      SQL Injection in user search e
1     unknown    open        XSS vulnerability found in log

i Showing 5/5 bugs
```

```
 ) ~/code/open-source/owasp/blt-cli ) on  master ?5
blt whoami
```

```
BLT
OWASP BLT — Bug Logging Tool

Username: unknown
Email:    unknown
```

tldr: *Week 9 handles BLT-Start + full landing page integration. Week 10 is entirely BLT-CLI in Rust; auth, bugs, leaderboard, jobs, timelogs, rewards, full flag system. The command reference table goes in Section 5 under a new BLT-CLI subsection alongside the screenshot you already have. That's the "CLI stuff" Donnie wanted combined into the proposal.*

6. Implementation Plan

Three phases. 12 weeks. Each phase ends at a GSoC evaluation checkpoint.

Phase 1 — Foundation & Core Submodules (Weeks 1–4)

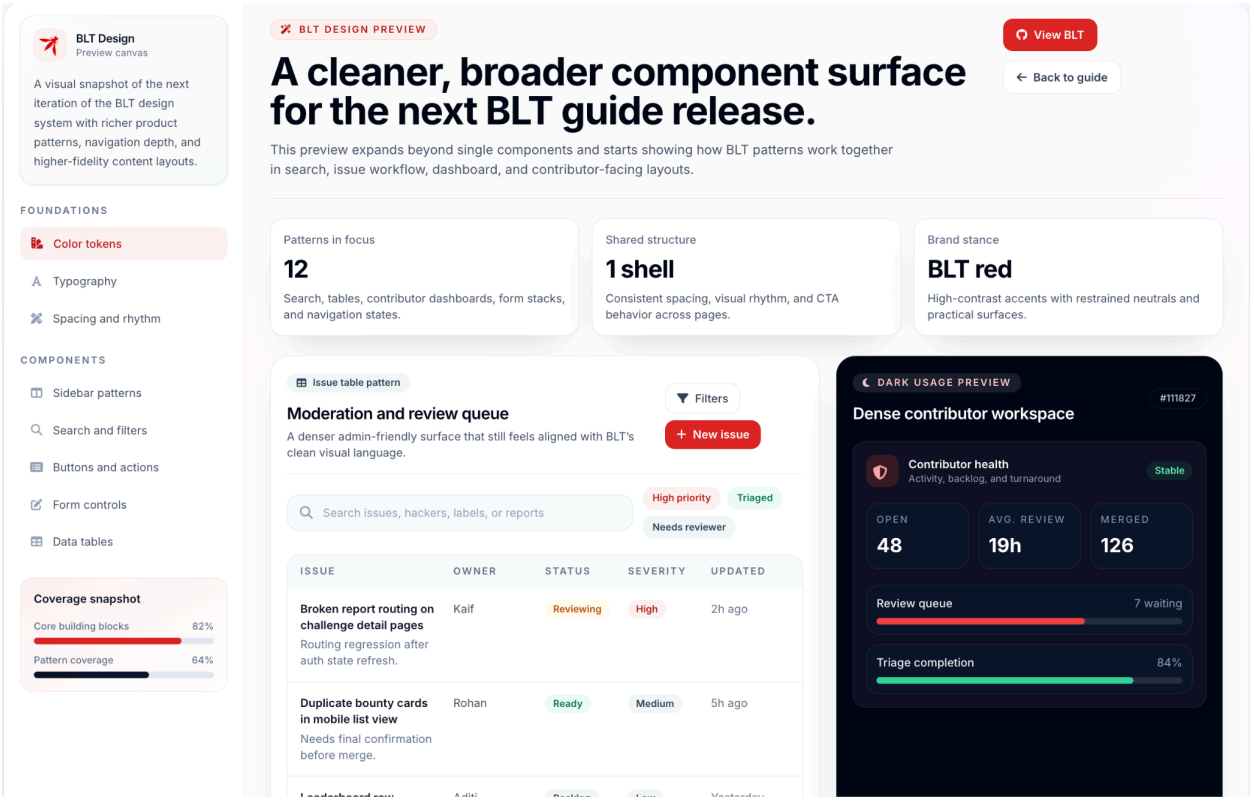
Days	Task
Day 1	D1 schema for anonymous reports `anon_reports` table. Fields: id, token, url, bug_type, severity, description, status, created_at. No user_id column, ever
Day 2	Worker: `POST /api/report` — validates input, generates random tracking token, writes to D1, returns token. No auth required
Day 3	Worker: `GET /api/report/:token` — returns report status by token only. No personal data exposed
Day 4	Frontend: report form — target URL, bug type selector, severity picker, description, optional file upload. HTMX wired to worker
Day 5	Frontend: token confirmation page — shown after submit. Track status page — enter token, see current status
Day 6	Deploy worker + frontend. Full end-to-end test — submit report, get token, check status, confirm D1 row has zero personal data
Day 7	Edge cases — missing fields, duplicate tokens, very long descriptions, file upload size limits. Fix everything

Week 2 — BLT-Design + BLT-Docs

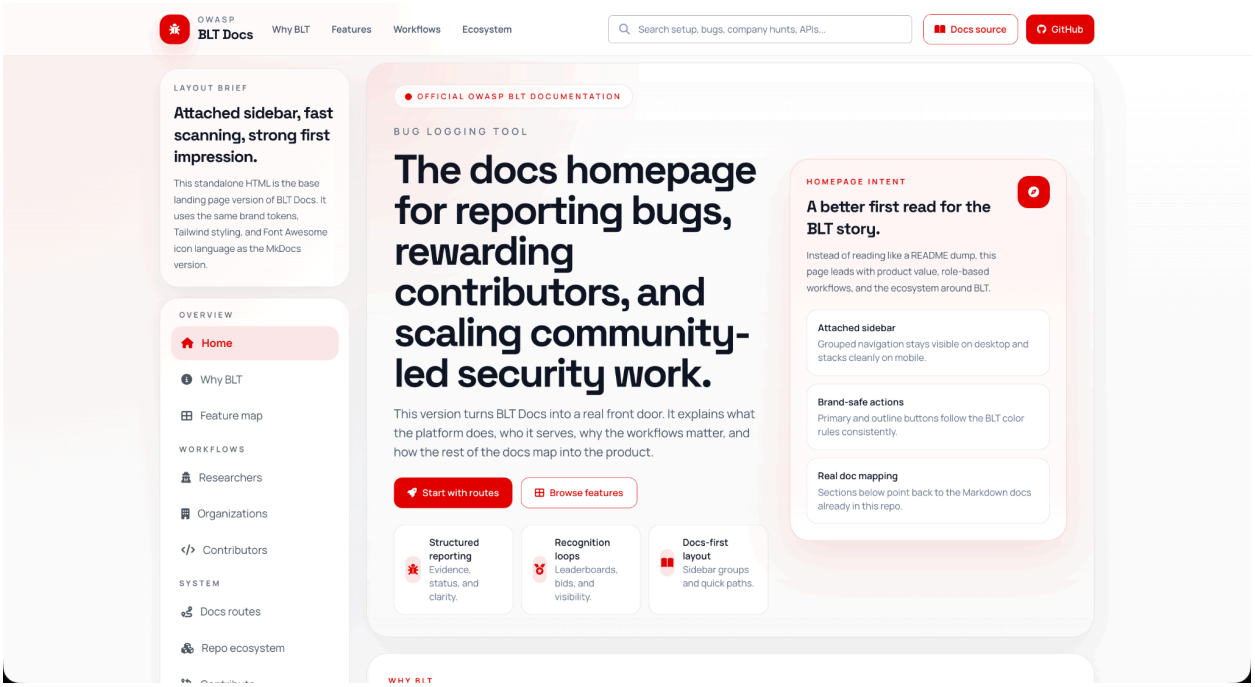
Days	Task
Day 1	Finalize D1 schema for all remaining submodules. Set up `wrangler.toml` with routes per worker. Confirm local dev works end to end
Day 2	Auth worker — JWT sign/verify, login + register endpoints, D1 users table. Needed for Timer-Web and other auth-gated submodules
Day 3	Shared CORS config across all workers. GitHub Actions pipeline auto deploy to GitHub Pages on push for each submodule
Day 4	BLT-Design: build core component library — buttons, cards, forms, nav, badges, dark mode. All plain HTML/CSS
Day 5	BLT-Design: typography scale, color system, spacing tokens. Publish to GitHub Pages
Day 6	BLT-Docs: scaffold architecture overview, submodule pattern guide, anonymous reporting deep-dive (this is the flagship, it gets its own doc page)
Day 7	BLT-Docs: contributing guide how to add a new submodule. Both BLT-Design and BLT-Docs live and linked from landing page

UI Mockup

BLT Design:



BLT Docs:

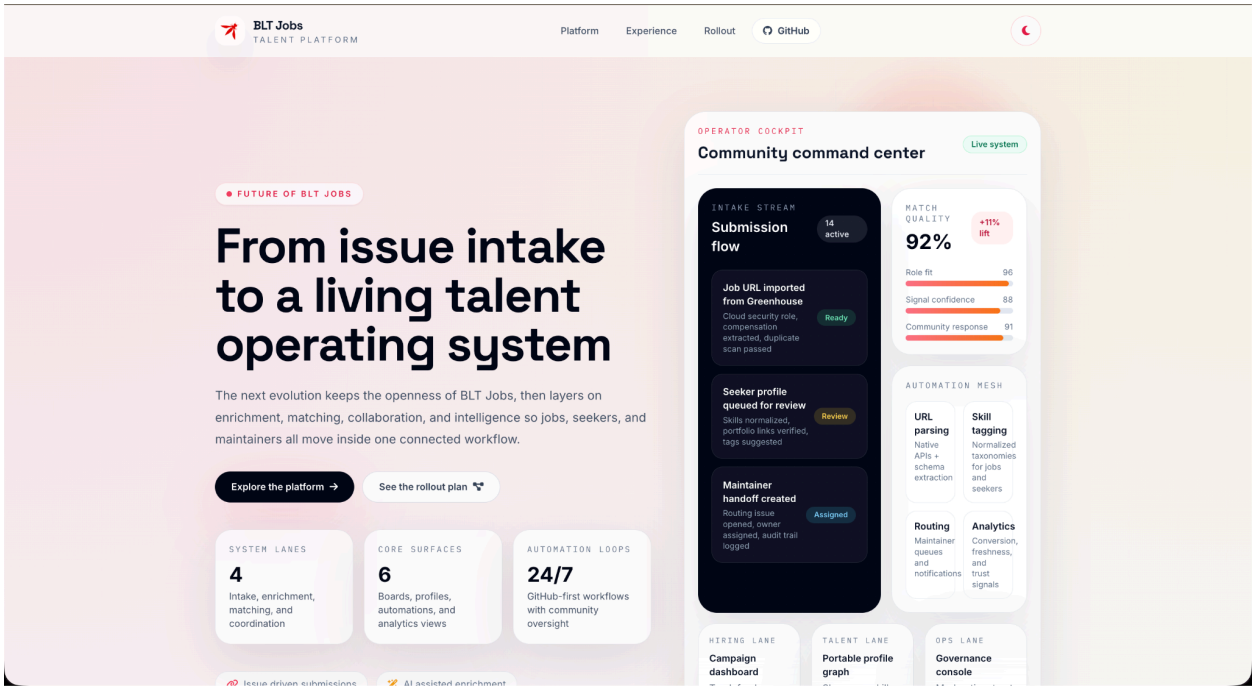


Week 3 — BLT-Jobs + BLT-Hackathons

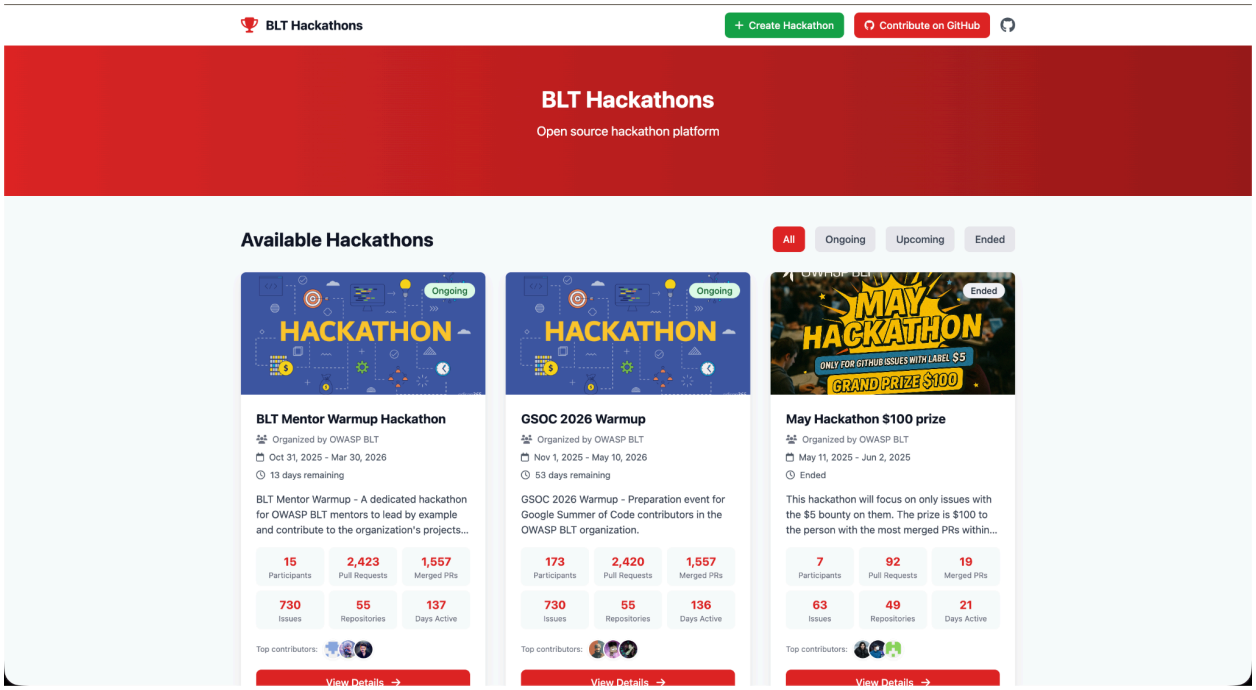
Days	Task
Day 1	BLT-Jobs: D1 schema, worker scaffold, CORS config
Day 2	BLT-Jobs: `GET /api/jobs` + `POST /api/jobs` worker endpoints. Input validation
Day 3	BLT-Jobs: frontend — job listings page, submission form, HTMX wired to worker
Day 4	BLT-Hackathons: D1 schema, worker scaffold, `GET /api/hackathons` + `POST /api/hackathons`
Day 5	BLT-Hackathons: frontend — listings page, status badges (upcoming/ongoing/ended), registration link
Day 6	Both submodules deployed to GitHub Pages. Workers deployed to Cloudflare
Day 7	Test all flows — job submission, hackathon listing, edge cases (empty state, missing fields). Fix anything broken

UI Mockup

BLT Jobs:



BLT Hackathon (will be extended with landing page):

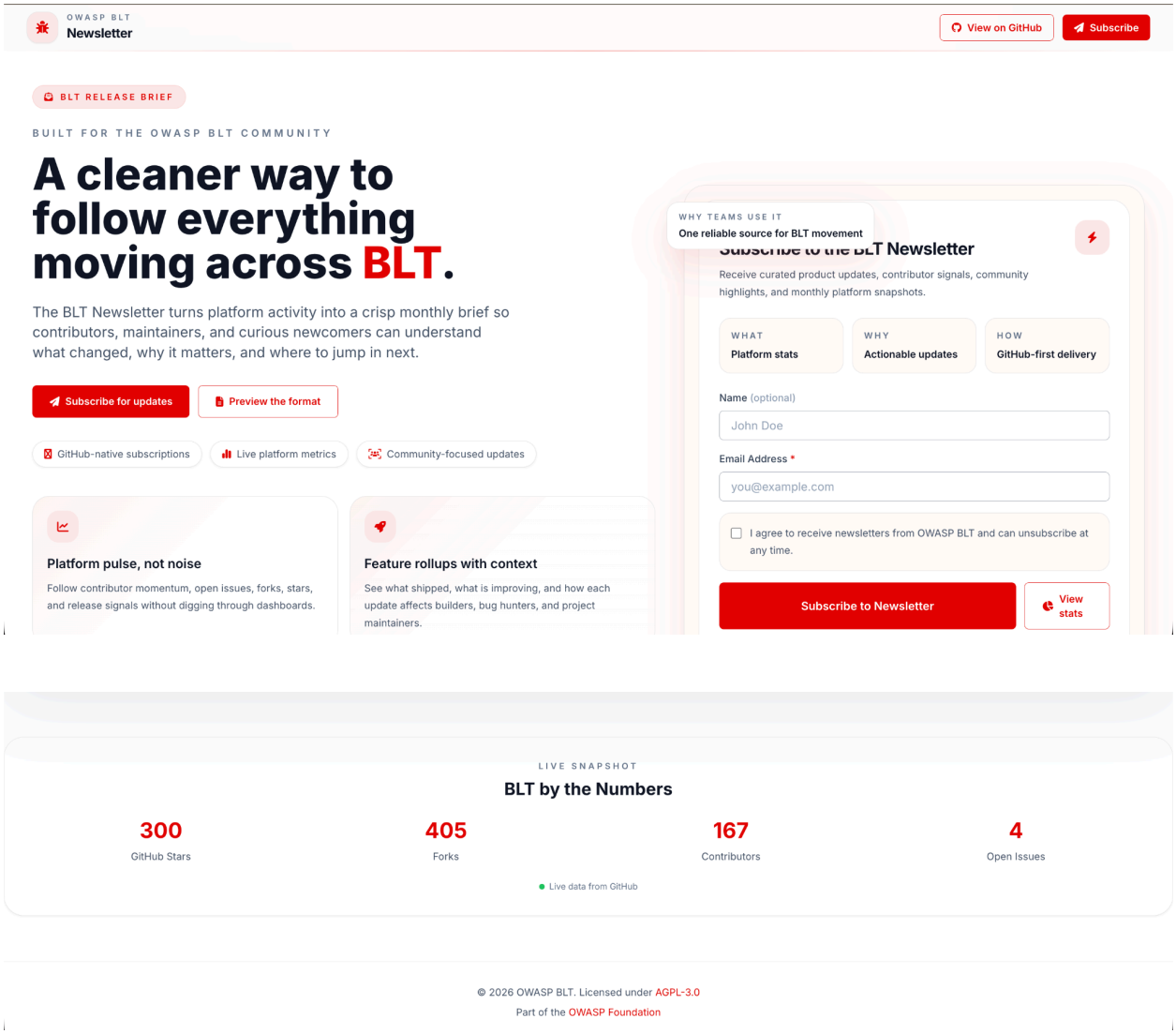


Week 4 — BLT-Newsletter

Days	Task
Day 1	BLT-Newsletter: D1 schema — subscribers table + issues table. Worker scaffold
Day 2	BLT-Newsletter: `POST /api/subscribe`, `DELETE /api/unsubscribe` worker endpoints
Day 3	BLT-Newsletter: `GET /api/issues` endpoint. Frontend — archive page, subscribe form
Day 4	BLT-Newsletter: deployed and live. Subscribe flow tested end to end
Day 5	Phase 1 integration pass — cross-linking between all submodules, consistent design, dark mode audit
Day 6	Fix anything that surfaced in integration pass
Day 7	Buffer — catch up on anything that slipped in Phase 1

Midterm evaluation checkpoint — *BLT-Design, BLT-Docs, BLT-Jobs, BLT-Hackathons, BLT-Newsletter all deployed and functional.*

UI Mockup



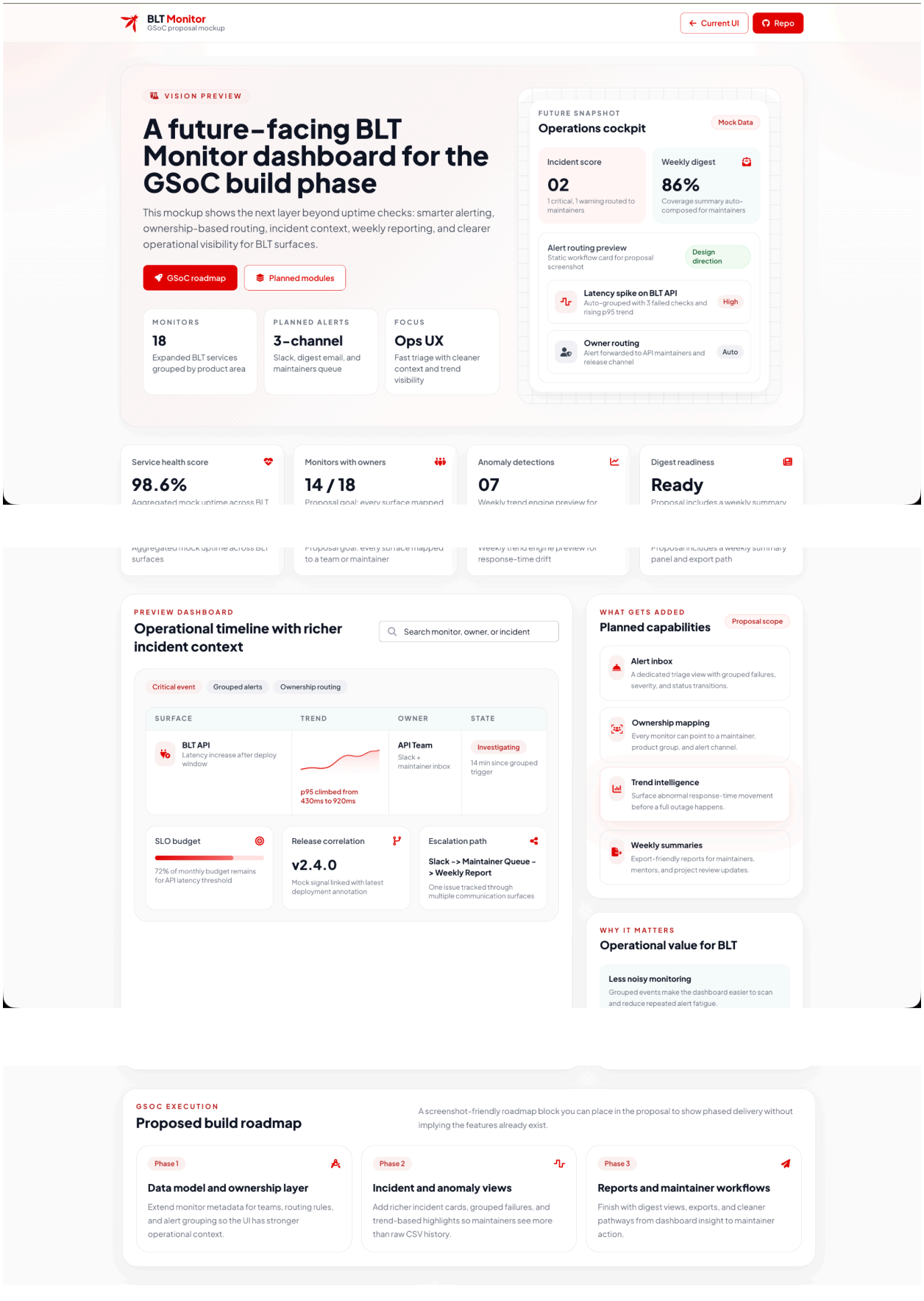
BLT-NEXT, GSOC 2026

Phase 2 — Developer Tools (Weeks 5–9)

Week 5 — BLT-Monitor

Days	Task
Day 1	D1 schema — `monitored_sites` + `uptime_logs`. Worker scaffold
Day 2	`GET /api/sites` + `POST /api/sites` endpoints. Add/list monitored URLs
Day 3	Cloudflare Cron Trigger — scheduled worker pings each site, writes result to `uptime_logs`
Day 4	`GET /api/sites/:id/logs` endpoint — fetch uptime history per site
Day 5	Frontend — dashboard UI, status badges, uptime percentage, last checked. HTMX polling for live-ish updates
Day 6	Deploy worker + frontend. Cron confirmed working in Cloudflare dashboard
Day 7	Test all states — site up, site down, site unreachable. Fix edge cases

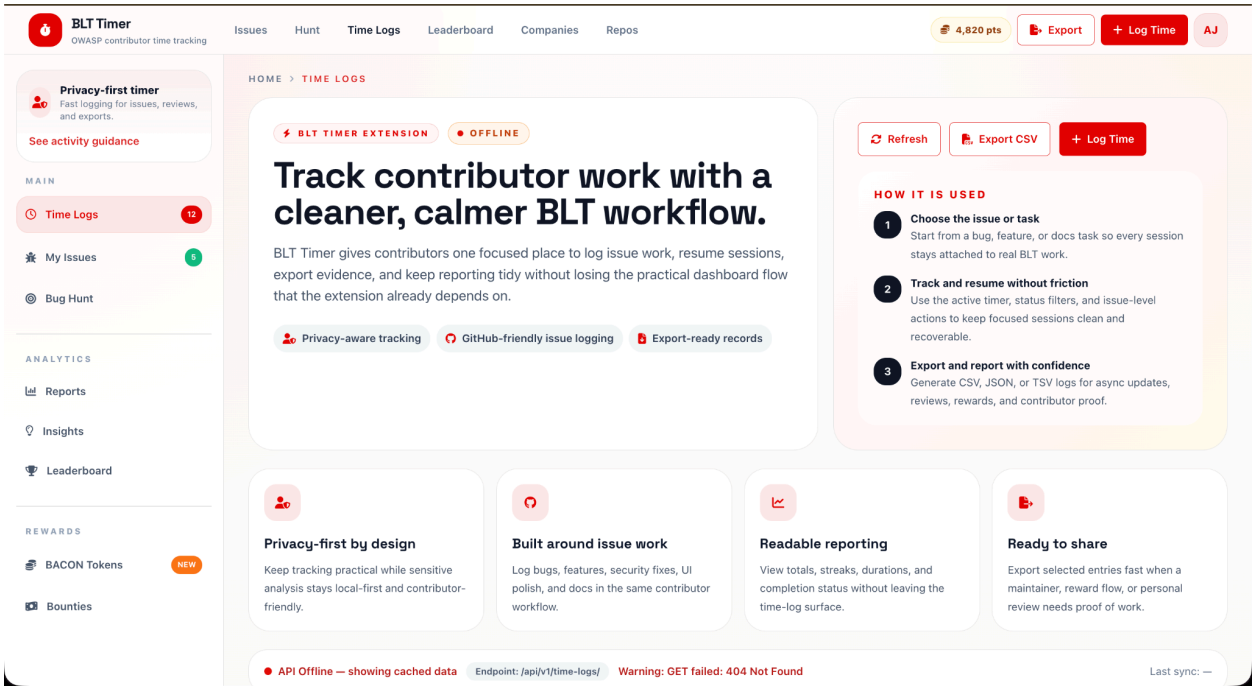
UI mockup:

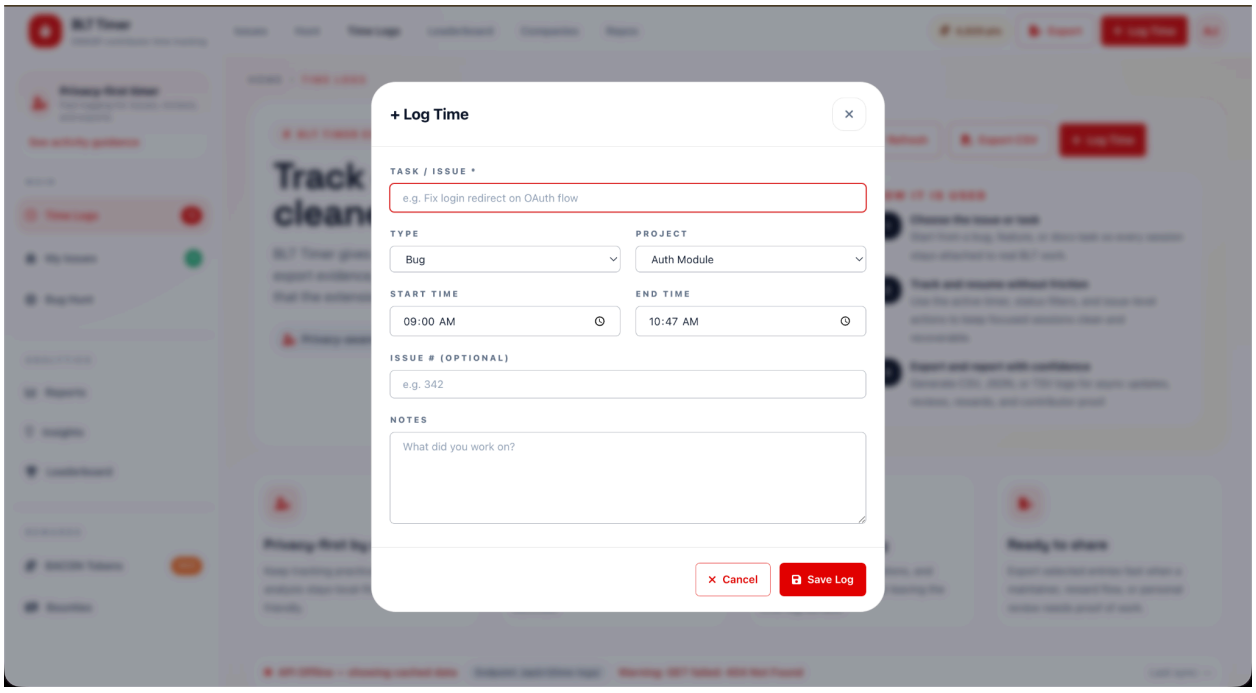

BLT Monitor
GSoC proposal mockup
Current UI
Repo
VISION PREVIEW
A future-facing BLT Monitor dashboard for the GSoC build phase
This mockup shows the next layer beyond uptime checks: smarter alerting, ownership-based routing, incident context, weekly reporting, and clearer operational visibility for BLT surfaces.
GSoC roadmap
Planned modules
MONITORS
18
Expanded BLT services grouped by product area
PLANNED ALERTS
3-channel
Slack, digest email, and maintainers queue
FOCUS
Ops UX
Fast triage with cleaner context and trend visibility
FUTURE SNAPSHOT
Operations cockpit
Mock Data
Incident score
02
1 critical, 1 warning routed to maintainers
Weekly digest
86%
Coverage summary auto-composed for maintainers
Alert routing preview
Static workflow card for proposal screenshot
Design direction
Latency spike on BLT API
Auto-grouped with 3 failed checks and rising p95 trend
High
Owner routing
Alert forwarded to API maintainers and release channel
Auto
Service health score
98.6%
Aggregated mock uptime across BLT surfaces
Monitors with owners
14 / 18
Proposal goal: every surface mapped to a team or maintainer
Anomaly detections
07
Weekly trend engine preview for response-time drift
Digest readiness
Ready
Proposal includes a weekly summary panel and export path
PREVIEW DASHBOARD
Operational timeline with richer incident context
Search monitor, owner, or incident
Critical event
Grouped alerts
Ownership routing
SURFACE
TREND
OWNER
STATE
BLT API
Latency increase after deploy window
p95 climbed from 430ms to 920ms
API Team
Slack + maintainer inbox
Investigating
14 min since grouped trigger
SLO budget
72% of monthly budget remains for API latency threshold
Release correlation
v2.4.0
Mock signal linked with latest deployment annotation
Escalation path
Slack -> Maintainer Queue -> Weekly Report
One issue tracked through multiple communication surfaces
WHAT GETS ADDED
Planned capabilities
Proposal scope
Alert inbox
A dedicated triage view with grouped failures, severity, and status transitions.
Ownership mapping
Every monitor can point to a maintainer, product group, and alert channel.
Trend intelligence
Surface abnormal response-time movement before a full outage happens.
Weekly summaries
Export-friendly reports for maintainers, mentors, and project review updates.
WHY IT MATTERS
Operational value for BLT
Less noisy monitoring
Grouped events make the dashboard easier to scan and reduce repeated alert fatigue.
GSOC EXECUTION
Proposed build roadmap
A screenshot-friendly roadmap block you can place in the proposal to show phased delivery without implying the features already exist.
Phase 1
Data model and ownership layer
Extend monitor metadata for teams, routing rules, and alert grouping so the UI has stronger operational context.
Phase 2
Incident and anomaly views
Add richer incident cards, grouped failures, and trend-based highlights so maintainers see more than raw CSV history.
Phase 3
Reports and maintainer workflows
Finish with digest views, exports, and cleaner pathways from dashboard insight to maintainer action.

Week 6 — BLT-Timer-Web

Days	Task
Day 1	D1 schema `timer_sessions`. Worker scaffold. Auth middleware wired
Day 2	`POST /api/sessions/start` + `POST /api/sessions/stop` endpoints. Auth-gated
Day 3	`GET /api/sessions` fetch session history per user
Day 4	Frontend: start/stop timer UI in vanilla JS. Timer logic client-side only
Day 5	Frontend: session history list, total time tracked, per-session labels
Day 6	Deploy and test. Confirm sessions save correctly per user via JWT
Day 7	Edge cases: duplicate starts, missing stops, very long sessions. Fix

UI Mockup

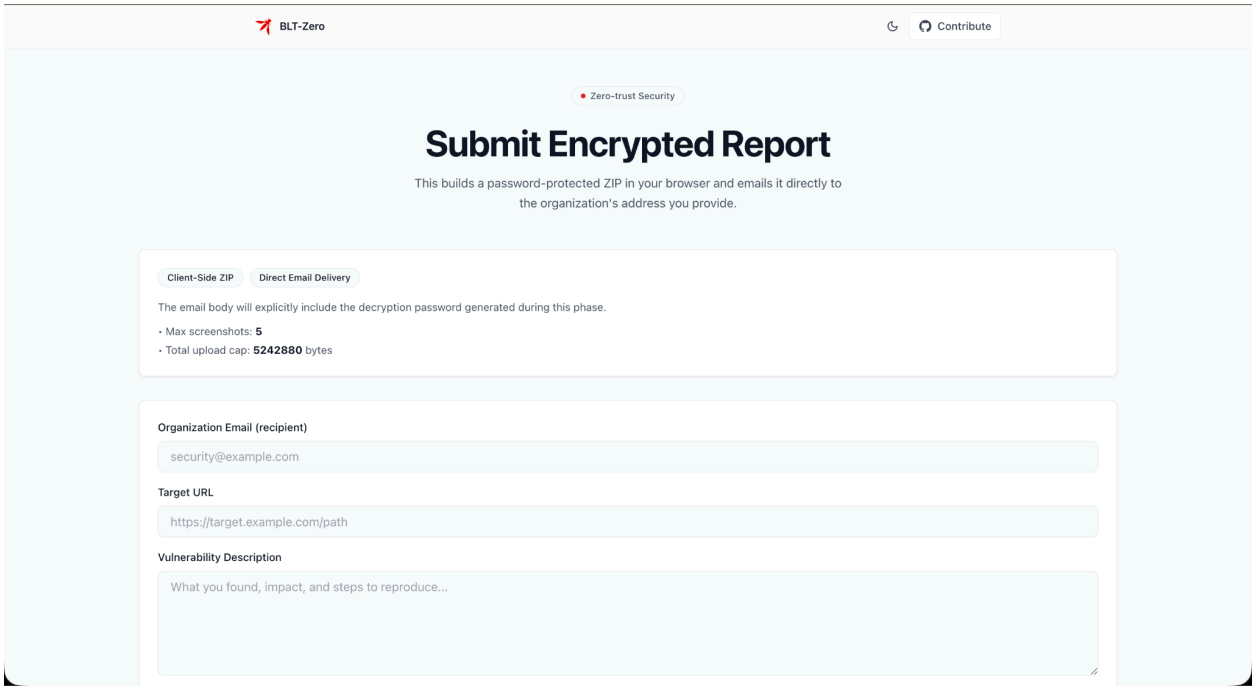

BLT Timer
OWASP contributor time tracking
Issues
Hunt
Time Logs
Leaderboard
Companies
Repos
4,820 pts
Export
+ Log Time
AJ
Privacy-first timer
Fast logging for issues, reviews, and exports.
See activity guidance
MAIN
Time Logs 12
My Issues 5
Bug Hunt
ANALYTICS
Reports
Insights
Leaderboard
REWARDS
BACON Tokens NEW
Bounties
HOME › TIME LOGS
BLT TIMER EXTENSION
OFFLINE
Track contributor work with a cleaner, calmer BLT workflow.
BLT Timer gives contributors one focused place to log issue work, resume sessions, export evidence, and keep reporting tidy without losing the practical dashboard flow that the extension already depends on.
Privacy-aware tracking
GitHub-friendly issue logging
Export-ready records
Refresh
Export CSV
+ Log Time
HOW IT IS USED
1 Choose the issue or task
Start from a bug, feature, or docs task so every session stays attached to real BLT work.
2 Track and resume without friction
Use the active timer, status filters, and issue-level actions to keep focused sessions clean and recoverable.
3 Export and report with confidence
Generate CSV, JSON, or TSV logs for async updates, reviews, rewards, and contributor proof.
Privacy-first by design
Keep tracking practical while sensitive analysis stays local-first and contributor-friendly.
Built around issue work
Log bugs, features, security fixes, UI polish, and docs in the same contributor workflow.
Readable reporting
View totals, streaks, durations, and completion status without leaving the time-log surface.
Ready to share
Export selected entries fast when a maintainer, reward flow, or personal review needs proof of work.
API Offline — showing cached data
Endpoint: /api/v1/time-logs/
Warning: GET failed: 404 Not Found
Last sync: —


+ Log Time
TASK / ISSUE *
e.g. Fix login redirect on OAuth flow
TYPE
Bug
PROJECT
Auth Module
START TIME
09:00 AM
END TIME
10:47 AM
ISSUE # (OPTIONAL)
e.g. 342
NOTES
What did you work on?
Cancel
Save Log

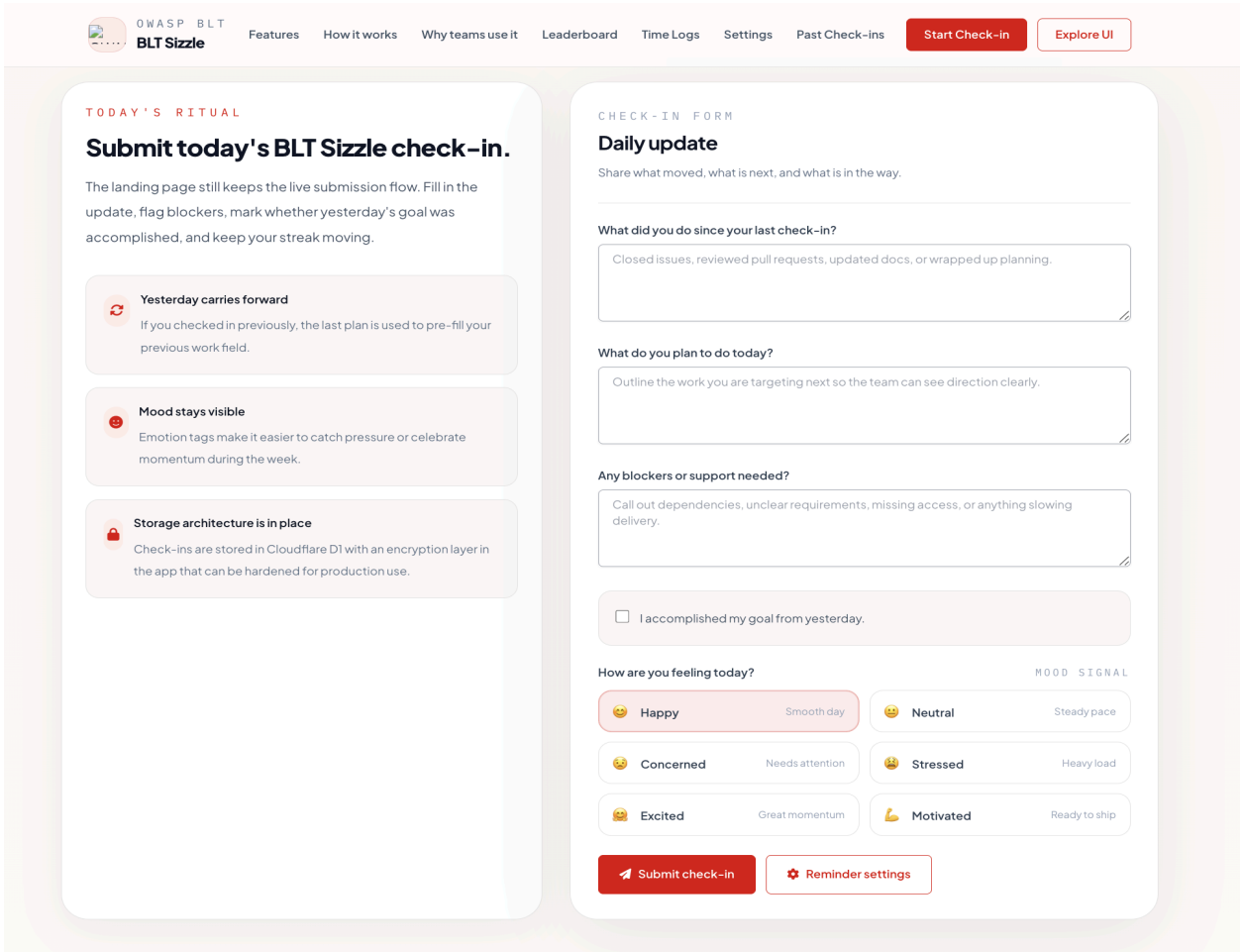
Week 7 — BLT-Sizzle + BLT-Zero

Days	Task
Day 1	BLT-Sizzle: worker scaffold. Logic to aggregate recent activity and rank by engagement from D1
Day 2	BLT-Sizzle: `GET /api/trending` endpoint. Frontend — live feed via HTMX polling every 30s
Day 3	BLT-Sizzle: deployed and tested. Confirm polling works without hammering the worker
Day 4	BLT-Zero: D1 schema `disclosures` table with status field. Worker scaffold
Day 5	BLT-Zero: `GET /api/disclosures` + `POST /api/disclosures` endpoints. Status tracking (draft/published/patched)
Day 6	BLT-Zero: frontend disclosure board, submission form, status badges
Day 7	Both deployed and tested. Cross-link from BLT-Next landing

UI Mockup



BLT Sizzle:

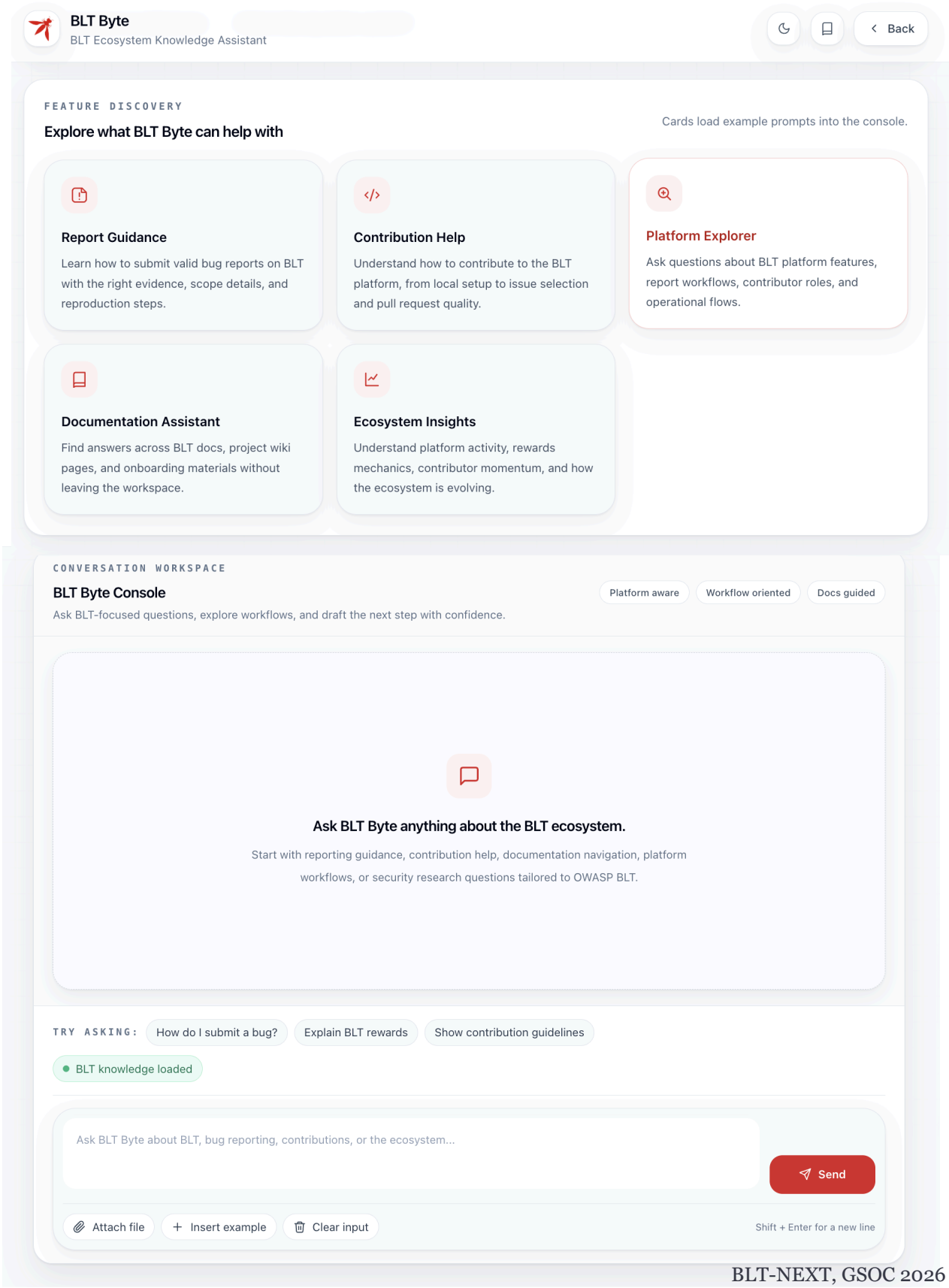


Week 8 — BLT-Byte + BLT-Panini

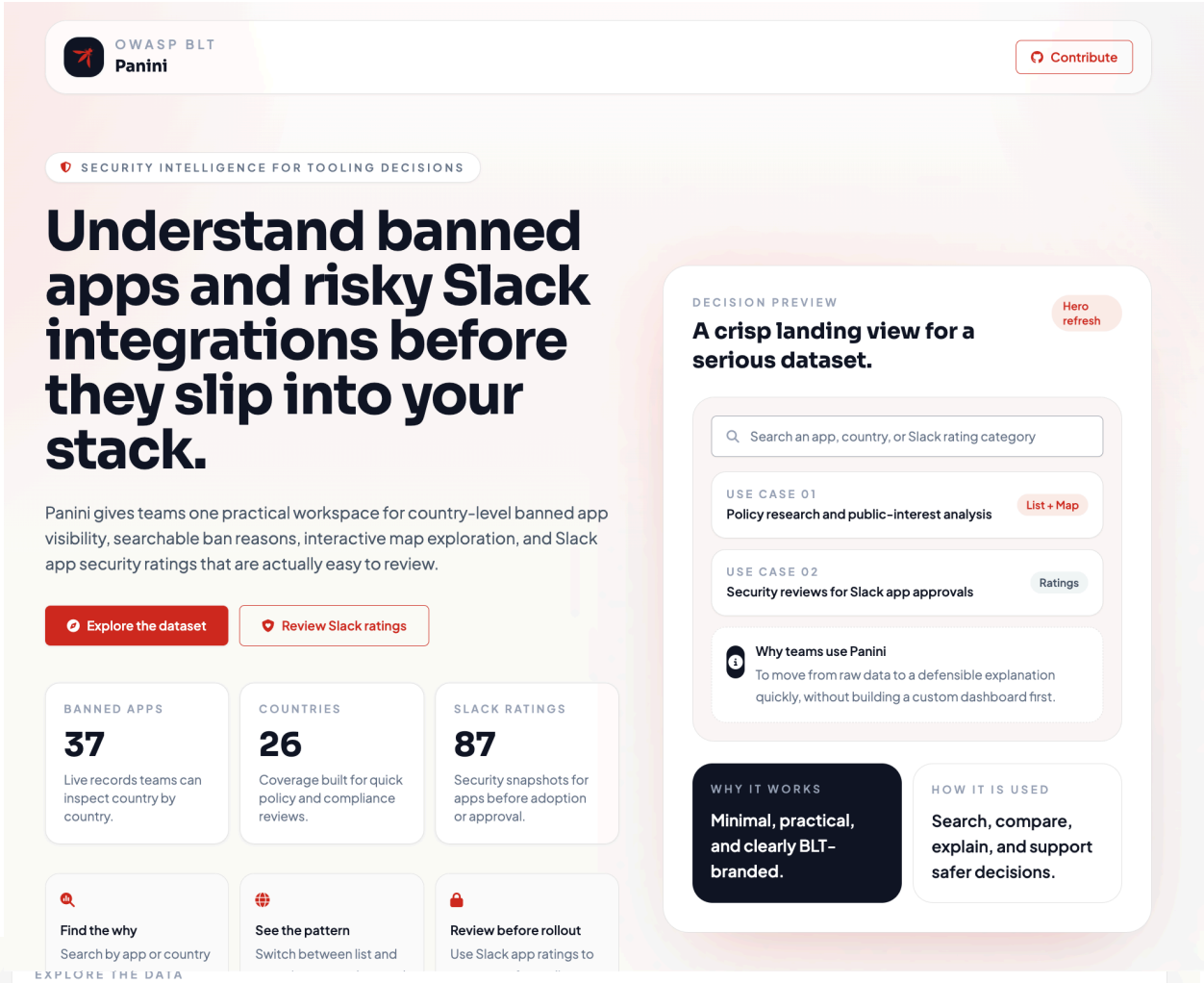
Days	Task
Day 1	BLT-Byte: content structure — markdown files for tips, rendered as static HTML at deploy time
Day 2	BLT-Byte: frontend — tips feed, category filter, tip detail page
Day 3	BLT-Byte: optional worker for reactions/voting. Deploy and test
Day 4	BLT-Panini: worker scaffold. Fetch + cache bug bounty listings from public sources
Day 5	BLT-Panini: D1 as cache layer — store fetched listings, serve from D1 to avoid hitting external APIs on every request
Day 6	BLT-Panini: frontend — bounty listings, filter by platform/reward range
Day 7	Both deployed and tested. Fix any caching issues with BLT-Panini

UI Mockup

BLT Byte



BLT Panini



EXPLORE THE DATA

Banned apps, map insights, and Slack app reviews in one place.

Use the list view for rapid scanning, map view for regional context, and Slack ratings for integration risk reviews.

List View | Map View

Slack App Ratings

Search by country or app name...

Banned Apps Results

EVIDENCE-FIRST VIEW

TikTok	INDIA	SOCIAL Security concerns and data privacy issues	29/06/2020 Details
PUBG Mobile	INDIA	GAMING Security concerns and addiction-related issues	02/09/2020 Details
Facebook	CHINA	SOCIAL Government censorship and internet regulations	01/07/2009 Details
WhatsApp	CHINA	MESSAGING Government censorship and national security concerns	25/09/2017 Details
LinkedIn	RUSSIA	SOCIAL Violation of data storage laws	17/11/2016 Details
Telegram	IRAN	MESSAGING Government censorship and national security concerns	30/04/2018 Details
Huawei AppGallery	UNITED STATES		
WeChat	INDIA		
UC Browser	INDIA		

BLT-NEXT, GSOC 2026

Week 9 — BLT-Start + Landing Page Integration

Days	Task
Day 1	BLT-Start: scaffold static site contributor onboarding hub. Sections: local setup, good first issues, PR guide, code review expectations, style guide
Day 2	BLT-Start: style guide pages button variants, input components, icon usage with Font Awesome 6. All rendered as live interactive examples, not just docs
Day 3	BLT-Start: mentor guide section how to label issues, set scope, respond to PRs. Link to `#project-blt` Slack and good first issues filter
Day 4	BLT-Start: deploy to GitHub Pages. Wire into BLT-Next landing page. Confirm all links, local setup instructions tested end to end
Day 5	Landing page integration pass — all submodules surfaced on BLT-Next home with live status, consistent nav, submodule cards linking correctly
Day 6	Dark mode audit across every submodule — fix inconsistencies. Mobile responsiveness pass at 375px
Day 7	Performance pass — confirm every page loads under 200ms. Fix anything above threshold

Week 10 — BLT-CLI (Rust)

Days	Task
Day 1	Scaffold Rust project `cargo init blt-cli`. Set up CLI framework with `clap`. Config system — `blt config set-key`, `blt config set-url`. Store in `~/.blt/config.toml`
Day 2	Auth commands — `blt auth login`, `blt auth logout`, `blt auth whoami`. JWT stored in `~/.blt/credentials.toml`. Talk to BLT-API auth worker
Day 3	Bug commands — `blt bugs list`, `blt bugs post "<title>"`, `blt bugs get <id>`. Wire to `/api/v1/bugs/`
Day 4	Leaderboard + profile commands — `blt leaderboard`, `blt profile`, `blt profile contributors`. Wire to `/api/v1/leaderboard/` and `/api/v1/profile/`
Day 5	Extended commands — `blt jobs list`, `blt orgs list`, `blt timelogs list`, `blt rewards`. Wire to `/api/v1/jobs/`, `/api/v1/organizations/`, `/api/v1/timelogs/`, `/api/rewards/`

Day 6	Output formatting — table view, JSON flag `--json`, color output with `colored` crate. `blt bugs list --json` for piping into other tools
Day 7	Test all commands against live BLT-API. Fix failures. Publish binary via GitHub Releases. Document all commands in BLT-Docs

Phase 2 checkpoint — *BLT-Monitor, BLT-Timer-Web, BLT-Sizzle, BLT-Zero, BLT-Byte, BLT-Panini, BLT Bytes, BLT-start, BLT CLI-Web all deployed and functional.*

Phase 3 — Polish, Integration & Docs (Weeks 10–12)

Week 10 — Landing Page Integration + Performance

Days	Task
Day 1–2	Wire all 12 submodules into BLT-Next landing page navigation. Submodule cards with live status pulled from each worker
Day 3	Dark mode audit across every submodule — fix inconsistencies
Day 4	Performance pass — check load times on all pages, identify anything above 200ms
Day 5	Fix performance issues. Optimize worker cold starts where possible
Day 6	Mobile responsiveness audit across all submodules

Day 7	Fix mobile issues. Final visual QA pass

Week 11 — Testing Pass

Days	Task
Day 1	Worker endpoint testing every route, every method, expected + unexpected inputs
Day 2	Auth flow testing login, JWT expiry, protected route rejection
Day 3	HTMX interaction testing every form, every dynamic load, every poll
Day 4	Empty states + error states what happens when D1 returns nothing, when worker throws, when network is slow
Day 5	Cross-browser testing Chrome, Firefox, Safari. Fix anything broken
Day 6	Fix everything that surfaced during the testing week
Day 7	Re-test fixes. Confirm everything is stable

Week 12 — Docs + Final Polish

Days	Task
Day 1–2	BLT-Docs: complete API reference for every worker endpoint across all 12 submodules
Day 3	BLT-Docs: update contributing guide with final submodule pattern. Add FAQ for common setup issues
Day 4	Final deployment check every submodule live, every link working, every worker responding
Day 5	Write end-of-GSoC summary for the BLT community, what was built, what's next
Day 6	Buffer last fixes, last doc updates
Day 7	Submit final GSoC evaluation. Done

BLT-NEXT, GSOC 2026

Timeline Summary

Week	Focus	Phase
1	Infrastructure, schema, auth, CI/CD	Phase 1
2	BLT-Design + BLT-Docs	Phase 1
3	BLT-Jobs + BLT-Hackathons	Phase 1
4	BLT-Newsletter + buffer	Phase 1
5	BLT-Monitor	Phase 2
6	BLT-Timer-Web	Phase 2
7	BLT-Sizzle + BLT-Zero	Phase 2
8	BLT-Byte + BLT-Panini	Phase 2
9	BLT-Start + BLT CLI	Phase 2
10	Landing page integration + performance (for all submodules)	Phase 3
11	Testing pass	Phase 3
12	Docs + final polish	Phase 3

7. Testing Strategy

No fluff here. Solo GSoC project, 12 weeks — the testing strategy needs to be realistic, not a list of things that sound good on paper but never happen.

Three layers: worker testing, frontend testing, integration testing.

Worker Testing

Every worker endpoint gets tested locally using `wrangler dev` before it touches Cloudflare. The test pattern is straightforward hit the endpoint with expected inputs, unexpected inputs, and missing inputs. Confirm the response shape is correct every time.

For D1 interactions specifically, every query gets tested against a local D1 instance first. Wrangler spins up a local SQLite-backed D1 — close enough to production to catch schema issues and query bugs before they land on the real database.

```
# Example test cases for BLT-Jobs worker
# Run against wrangler dev at http://localhost:8787
test_cases = [
    {
        "desc": "GET /api/jobs — returns list",
        "method": "GET",
        "url": "/api/jobs",
        "expect_status": 200,
        "expect_keys": ["id", "title", "company", "url", "created_at"],
    },
    {
        "desc": "POST /api/jobs — creates job",
        "method": "POST",
        "url": "/api/jobs",
        "body": {
            "title": "Security Engineer",
            "company": "OWASP",
            "url": "https://example.com",
        },
        "expect_status": 201,
        "expect_keys": ["success"],
    },
    {
        "desc": "POST /api/jobs — missing company",
        "method": "POST",
        "url": "/api/jobs",
        "body": {"title": "Security Engineer"},
        "expect_status": 400,
        "expect_keys": ["error"],
    },
    {
        "desc": "DELETE /api/jobs — not allowed",
        "method": "DELETE",
        "url": "/api/jobs",
        "expect_status": 404,
    },
]
```

Same pattern repeated for every submodule worker. Not glamorous but it catches 90% of bugs before they go live.

Auth Testing

Auth gets its own test pass because a broken auth worker breaks everything that depends on it.

Test Case	Expected Outcome
Login with valid credentials	Returns JWT, 200
Login with wrong password	Returns 401
Login with missing fields	Returns 400
Request protected route with valid JWT	Returns data, 200
Request protected route with expired JWT	Returns 401
Request protected route with no token	Returns 401
Request protected route with tampered JWT	Returns 401

JWT expiry specifically — test that a token signed with `exp: now - 1` gets rejected. That's the one auth bug that's easy to miss and very bad in production.

Frontend Testing

No test framework for the frontend — that would mean a build step, which defeats the whole point of the stack. Instead, manual testing against a checklist for each submodule.

Every submodule gets checked for:

Check	What It Covers
Page loads without JS	Progressive enhancement baseline
HTMX requests fire correctly	Network tab — confirm requests hit the right worker URL
Form submission — valid input	Data appears in D1, UI updates correctly
Form submission — invalid input	Error message shown, nothing written to D1
Empty state	What renders when D1 returns zero rows
Error state	What renders when worker returns 4xx or 5xx
Dark mode	Every element renders correctly in dark mode
Mobile at 375px	Nothing breaks, nothing overflows

BLT-Monitor Specific Testing

BLT-Monitor has a cron worker — that needs a slightly different approach. Cloudflare lets you trigger a cron manually via the dashboard during development. The test flow:

- Add a test site that's guaranteed to be up (e.g. `https://cloudflare.com`)
- Add a test site that's guaranteed to be down (a non-existent URL)
- Trigger cron manually
- Confirm `uptime_logs` has two new rows — one `is_up = 1`, one `is_up = 0`
- Confirm `monitored_sites` status field updated correctly for both

Integration Testing

End of each phase — before moving to the next one — run a full integration pass:

Phase 1 integration checklist:

- All Phase 1 submodules load from BLT-Next landing page
- CORS — every worker accepts requests from GitHub Pages origin, rejects everything else
- Dark mode consistent across BLT-Design, BLT-Docs, BLT-Jobs, BLT-Hackathons, BLT-Newsletter
- All GitHub Actions pipelines green — push to main deploys correctly

Phase 2 integration checklist:

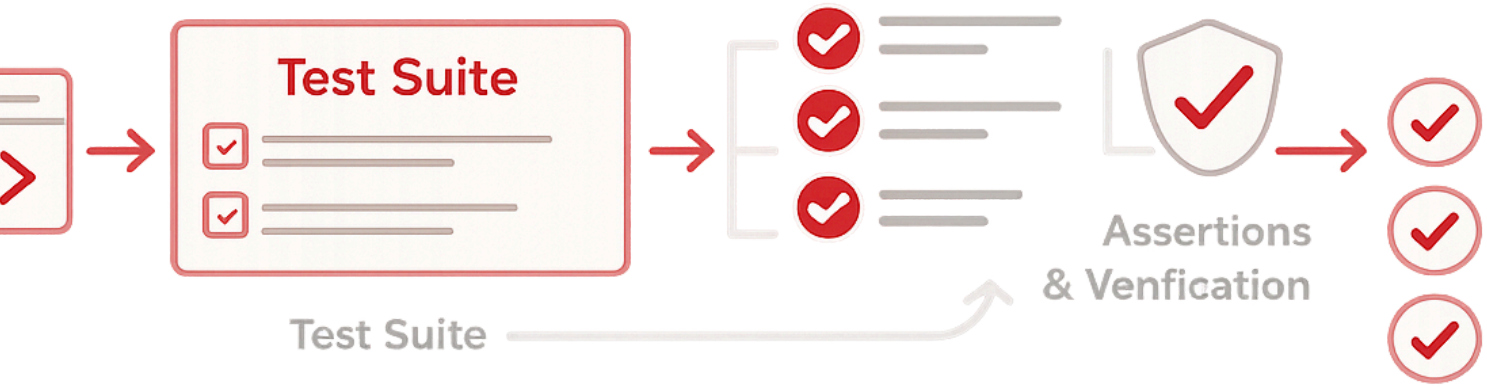
- All Phase 2 submodules linked and loading
- BLT-Monitor cron running on schedule in production
- Auth token from BLT-Next core works correctly in BLT-Timer-Web (cross-submodule auth)

- BLT-Docs updated with all Phase 2 submodule API references

Final integration checklist (Week 11):

- Every submodule live at its GitHub Pages URL
- Every worker responding at its `workers.dev` URL
- Every form submission writes correctly to D1
- Every JS interaction works in Chrome, Firefox, Safari
- No broken links between submodules
- BLT-Next landing page surfaces all 12 submodules correctly

tldr: *workers get tested locally with `wrangler dev` before deploy. Auth gets its own dedicated test pass. Frontend testing is manual but structured, same checklist for every submodule. Integration testing happens at the end of each phase, not just at the end of the project.*



8. Documentation Plan

Documentation isn't an afterthought here, it's a deliverable. BLT-Next is supposed to be contributor-friendly by design. That only works if someone can land on the repo, understand what's going on, and make a contribution without needing to ask Donnie on Slack.

Three types of docs: architecture docs, API reference, and contributor guides. All of it lives in BLT-Docs — the submodule built in Week 2.

Architecture Docs

Written once in Week 2, updated as a submodule ship. Covers the big picture why the stack is what it is, how the pieces connect, and what the standard submodule pattern looks like.

Document	What It Covers	When Written
Architecture Overview	Three-layer diagram, GitHub Pages + Workers + D1 explained, design decisions	Week 2
Submodule Pattern	Step-by-step: what every submodule needs, folder structure, wrangler config, GitHub Pages setup	Week 2
Auth Flow	JWT lifecycle, how frontend stores token, how workers validate it	Week 2

D1 Schema Reference	Full schema for all submodules, column types, relationships	Week 1 (schema) + Week 2 (docs)
Deployment Guide	How to deploy a worker, how GitHub Actions handles Pages deploys, environment variables	Week 2

API Reference

Every worker endpoint is documented. Written as each submodule ships — not left for the end.

Format for every endpoint:

GET /api/jobs

Returns a list of active job postings ordered by newest first.

Query params:

limit (optional) — number of results, default 50

Response 200:

Submodule	Endpoints Documented	Written In
Auth	`/api/login`, `/api/register`	Week 1
BLT-Jobs	`/api/jobs` GET + POST	Week 3
BLT-Hackathons	`/api/hackathons` GET + POST	Week 3
BLT-Newsletter	`/api/subscribe`, `/api/unsubscribe`, `/api/issues`	Week 4
BLT-Monitor	`/api/sites` GET + POST, `/api/sites/:id/logs`	Week 5
BLT-Timer-Web	`/api/sessions/start`, `/api/sessions/stop`, `/api/sessions`	Week 6
BLT-Sizzle	`/api/trending`	Week 7
BLT-Zero	`/api/disclosures` GET + POST	Week 7
BLT-Byte	`/api/tips` (if worker added)	Week 8
BLT-Panini	`/api/bounties`	Week 8
BLT-Vanish-Web	`/api/requests` POST, `/api/requests/:id` GET	Week 9

Full API reference pass in Week 12 — fill any gaps, confirm every endpoint is documented accurately against what actually shipped.

Contributor Guides

This is the most important part. The whole point of BLT-Next's architecture is low contributor friction. The docs need to make good on that promise.

"How to add a new submodule" — written in Week 2, the most critical guide. Walks through:

1. Create a new repo under OWASP-BLT organisation

2. Copy the submodule template (folder structure, base HTML, wrangler.toml)

3. Add your worker to workers/ with the standard pattern

4. Add your D1 tables to schema.sql

5. Run wrangler dev — confirm local works

6. Add GitHub Actions workflow for auto-deploy

7. Add your submodule to BLT-Next landing page

8. Open a PR — that's it

No Django setup. No virtualenv. No migrations. The guide should be short enough that a first-time contributor reads it in five minutes and knows exactly what to do.

"How to contribute to an existing submodule" — even simpler. Open the HTML file, make your change, push. The GitHub Action handles the rest.

"Local development setup" — covers:

- Installing wrangler
- `npm run dev` to start local server
- `npm run db:init` to seed local D1
- How to test worker endpoints locally

README Standards

Every submodule repo gets a README that follows the same template:

What Doesn't Get Documented

Being honest some things are out of scope for GSoC documentation:

- Internal Cloudflare infra details — environment variables, account IDs, D1 database IDs. Those live in the Cloudflare dashboard, not in public docs.
- Submodules not in the priority list — BLT-CVE, BLT-Toasty etc. get documented when they get built, not during GSoC.
- Video walkthroughs or interactive tutorials — out of scope, text docs are sufficient.

tldr: *BLT-Docs is the single source of truth. Architecture docs written in Week 2, API reference written as each submodule ships, contributor guides written early so they're useful during GSoC not just after. Every submodule README follows the same short template. The goal is a new contributor makes their first PR without asking anyone for help.*

9. Why Me?

I'm not going to say I'm passionate about open source or that this project aligns with my goals. Here's just what's actually true.

I've been contributing to BLT since November 2025. 55+ merged PRs across 13+ repos. I know the codebase, I know the architecture, I know where the rough edges are. This isn't me reading the README and making a plan; this is me already knowing what needs to be built and having context on most of it from real contributions.

The specific thing that matters for BLT-Next: I've worked on both sides of the exact stack this project is built on.

On the Django side; BLT Core contributions. Leaderboard integrity fixes, dark mode, deprecation fixes, organization bugs, settings cleanup. I know what the monolith looks like from the inside, what it does well, what makes it painful to work with.

On the Cloudflare Workers side; BLT-API is where I've spent the most time. Full D1 migration, bugs schema, user schema, 2FA email auth with Mailgun, domain routing, worker deployment setup. The exact layer BLT-Next is built on. I didn't read about this stack, I built production features on it.

On BLT-Next directly; dark mode (#15) and design update with logo (#9), both merged. I've already touched the repo, I know the current state of it.

tldr: *BLT-Next sits exactly at the intersection of Django knowledge (to understand what we're replacing) and Cloudflare Workers knowledge (to build what replaces it). That's not a coincidence; that's just where my contributions have been.*

I've also had real architectural conversations with Donnie about where BLT is going. Not just "I read the issues" actual back and forth about D1 as a single source of truth, about making BLT-API more modular, about the tradeoffs of the Cloudflare approach. I understand the vision, not just the task list.

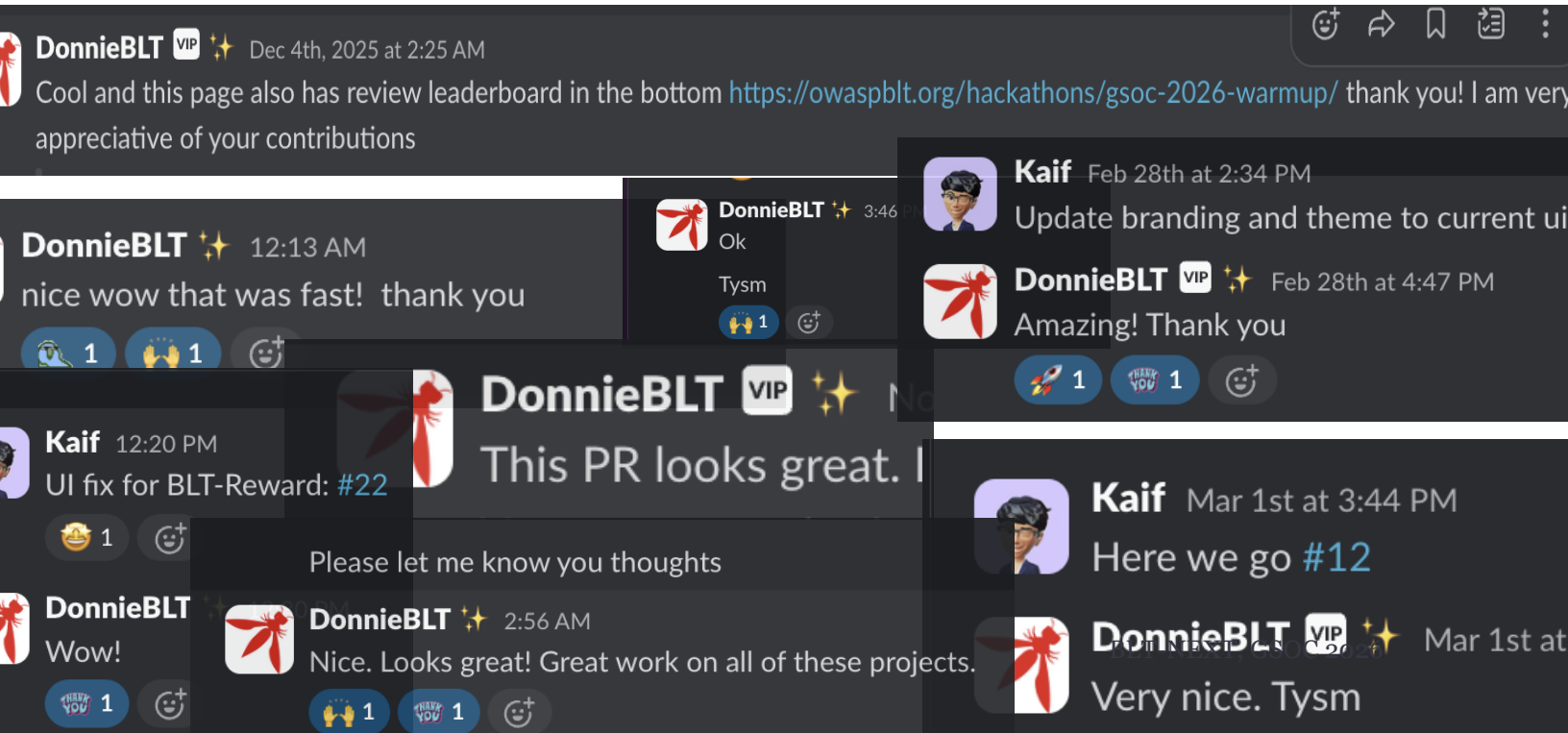
10. Other Commitments

I'm a 3rd year B.Tech student at Aliah University, batch 2024–2027. GSoC runs during my summer break with no exams, no coursework clashing with the program timeline.

I'm not applying to any other GSoC organizations. BLT-Next is the one project I want to work on.

Expected weekly availability: 40+ hours. I've been contributing to BLT at a serious pace since November while managing university; during GSoC with no academic load that pace goes up, not down.

Timezone: IST (UTC +5:30). I'm available for syncs with Donnie and the OWASP community during reasonable overlap hours.



LLM Usage Considerations

I used Claude (Anthropic) during the writing of this proposal. Specifically for structuring sections, drafting technical explanations, and building the interactive UI png. I reviewed content generated against the actual implementation. The implementation plan, scope decisions, and architectural choices reflect real conversations with Donnie(The maintainer of BLT)and real experience from 55+ PRs across the BLT ecosystem, not AI output. The proposal was written collaboratively with AI assistance(As I'm from India and English is my secondary language,in order to avoid grammatical error)but every decision in it is mine.

LLMs were used for support, providing a perfect template for section and editorial purposes in this work. All outputs were reviewed and validated by the contributor to ensure accuracy and originality.

11. References & Links

Resource	Link
BLT-Next Repository	https://github.com/OWASP-BLT/BLT-Next
OWASP BLT GitHub	https://github.com/OWASP-BLT
My GitHub	https://github.com/mdkaifansari04
Cloudflare Python Workers Docs	https://developers.cloudflare.com/workers/languages/python
Cloudflare D1 Docs	https://developers.cloudflare.com/d1
HTMX Docs	https://htmx.org/docs
Wrangler CLI Docs	https://developers.cloudflare.com/workers/wrangler
GSoC 2026 OWASP Project Page	https://owasp.org/www-community/initiatives/gsoc